Integer-Exact Predictive Control for Container Auto-Scaling: Measuring the Cost of Relaxation and the Horizon–Dead-Time Law

Atharva Bandekar

School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, India

Abstract—Container auto-scalers decide how many replicas to run, and that decision is an integer. Published predictive controllers nevertheless solve a relaxed, real-valued problem and round the result, and they rarely state how the forecast horizon should be chosen relative to the time a replica takes to become ready. This paper examines both choices experimentally on a discrete-event pool simulator driven by five synthetic workload families and by 10 days of the 1998 World Cup web-server log at two scalings. Every controller is run with two bracketing predictors, perfect foresight and persistence, so that controller effects are separated from predictor quality, and on the recorded traffic also with a temporal convolutional forecaster trained on its earlier days. Four findings follow. First, the forecast horizon must cover the actuation dead time or prediction is actively harmful: averaged over the 5 cold-start settings for which both sides of the boundary were sampled, violations fall from 2.92 % to 0.27 % as the horizon crosses $H\Delta t = \tau c$, and below that threshold the predictive controller is up to 3.69 percentage points worse than a reactive baseline, while above it it is up to 3.58 points better; on the recorded traffic at the scalings used, the reactive baseline never violated, so the transition could not be tested there. Second, the integer problem can be solved to global optimality rather than approximated. The dead time confines the tracking-relevant decisions to the first $H - \tau + 1$ steps of the horizon and leaves the remainder separable. This cuts the search from $(2\Delta max+1)^H$ to 4 nested integer choices, and the resulting solver matched exhaustive enumeration in 400 of 400 brute-forced plant states; completing the horizon greedily instead returned an optimal first action in only 354 of them, and never a better one. Third, relaxing integrality is not free: the relaxed-and-rounded controller disagrees with the exact solution on 18.8–63.6 % of control ticks and, at an aggressive utilisation target of 0.85, reaches 16.18 % violations against 2.86 % for the exact controller. Exact solution costs at most 17.5 ms per tick in the mean and 34.3 ms in the worst single solve observed, under 0.69 % of the control interval. Fourth, the trained forecaster does not sit between the brackets as the protocol assumes: it lies inside a non-degenerate bracket on 6 of 23 closed-loop comparisons. Its upward forecast bias reproduces the direction of its capacity-side effects in 14 of 14 cases but not its reduction of the rounded controller's churn. These results come from simulation, not from a deployed system; every value reported is measured, and none is estimated or illustrative.

Index Terms—Container auto-scaling, model predictive control, integer programming, actuation dead time, cold start, discrete-event simulation, Docker, resource management.

I. INTRODUCTION

A. Background

A container pool serves traffic with some number of identical replicas, and an auto-scaler adjusts that number as demand moves. The decision is discrete: a pool runs three replicas or four, never 3.4. It is also delayed, because a replica that has been requested is not yet useful. Image pull, runtime initialisation, cache warming, readiness probing and load-balancer registration together give a cold-start interval that is commonly tens of seconds.

Threshold auto-scalers, of which the Kubernetes Horizontal Pod Autoscaler is the canonical example, act only after utilisation has crossed a configured target. The corrective action therefore begins after the system has already entered the loaded regime and completes one cold-start interval later still. Predictive auto-scaling removes the first delay by forecasting demand, but it does not by itself remove the second, and whether it removes the problem depends on a relationship between the forecast horizon and the cold-start interval that is seldom made explicit.

B. Research Gap

Three gaps motivate this work, and all three concern the controller rather than the predictor.

The first is integrality. Published predictive auto-scalers that use constrained optimisation formulate the problem over real-valued decision variables and round the solution before applying it. The rounding step is reported as an implementation detail and its cost is not measured. Independent evidence suggests the cost is not negligible: in a recent benchmark of reinforcement-learning controllers, discrete-action algorithms outperformed continuous-action ones, an outcome the authors attribute to the mismatch introduced when continuous outputs are bucketed into integer replica changes. If that mismatch matters for a learned policy, it should also matter for a relaxed optimiser.

The second is tractability, and it is the reason the first gap has stayed open. Enforcing integrality on a receding-horizon problem appears to demand a search over $(2\Delta max + 1)$ raised to the power of the horizon length, which is why the usual response is either to relax the constraint or to enumerate the first action and complete the horizon with a heuristic. The second response is often described as exact on the grounds that only the first action is applied. That reasoning does not hold: the first action is chosen by comparing the costs of completed plans, so a biased completion biases the comparison. Whether an exact integer solve is affordable at container-scaling rates has not, as far as this survey found, been settled either way.

The third is horizon selection. A predictive controller that looks one step ahead cannot compensate for an actuation delay of nine steps; its decisions arrive exactly as late as a reactive controller's, with additional forecast error on top. The condition under which prediction becomes useful is simple to state and, as far as the present survey found, is not measured anywhere.

A worked example

A concrete instance makes the condition tangible. Suppose a pool of eight replicas is serving 260 requests per second at a 65 % target, demand doubles over the next thirty seconds, and the container image takes 45 s to become ready. The requirement will rise to roughly sixteen replicas. A reactive controller observes the rise only as utilisation crosses its target, issues the

scale-up at that moment, and the new replicas begin serving 45 s later, by which point the pool has been under-provisioned for the entire rise and part of the plateau.

A predictive controller with a one-step horizon is no better. It forecasts five seconds ahead, which tells it nothing about a transition that completes in thirty, and its scale-up lands at essentially the same moment as the reactive one, now with forecast error added. Only when the horizon reaches nine ticks does the forecast span the actuation delay, and only then can the controller issue at t the action whose effect arrives at t + 45 s, when the demand does. The measurements in Section V-A are this example repeated across six delays and ten horizons, and they show the transition occurring where this reasoning predicts.

C. Contributions

This paper makes five contributions, each supported by measurements reported in Section V.

- A measurement of the horizon–dead-time law. Sweeping the cold-start interval and the forecast horizon independently shows a sharp transition in service-level attainment that coincides with the condition $H\Delta t \geq \tau c$. Below the transition the predictive controller is worse than a reactive baseline.
- A tractable exact solve, with its structural argument and its verification. Because an action issued now does not reach the pool for τ ticks, only the first $H - \tau + 1$ actions of the horizon can influence any tracking term inside it, and the remaining actions enter the cost through a separable chain minimised by backward recursion. Together these cut the search to a size that can be enumerated, and the resulting solver is checked against exhaustive enumeration rather than asserted to be correct.
- A measurement of the cost of relaxation, and of the cost of approximating the solve. The exact controller is compared against an otherwise identical relaxed-and-rounded controller across five pool sizes, isolating the effect of the integrality constraint alone, and separately against a solver that enumerates first actions under a greedy horizon completion, isolating the effect of solving the integer problem only approximately.
- An evaluation protocol that separates controller effects from predictor quality by bracketing every controller between a perfect-foresight predictor and a persistence predictor, so that conclusions holding under both brackets are attributable to the control law.
- A test of the protocol and of the findings on recorded traffic. A temporal convolutional forecaster is trained on ten days of the 1998 World Cup web-site access log, compared against recurrent, attention-based and statistical baselines, and placed in the loop between the two brackets; the earlier findings are then re-tested on the recorded trace, and those that do not survive are reported as not surviving.

D. Organisation

Section II reviews existing auto-scaling systems and groups the recent literature by the mechanism each study adopts. Section III develops the controller. Section IV describes the experimental setup, the simulated plant and the workloads. Section V reports results, ending in Section V-Q with the recorded trace and the trained forecaster. Section VI discusses them, Section VII states the limitations, several of which are substantial, and Section VIII concludes.

II. EXISTING SYSTEMS AND RELATED WORK

A. Threshold-Based Auto-Scaling in Production Orchestrators

The auto-scalers actually deployed at scale are reactive and rule-based. The Kubernetes Horizontal Pod Autoscaler samples an aggregate utilisation metric on a fixed interval and sets the replica count to the current count scaled by the ratio of observed to target utilisation, subject to a tolerance band and a scale-down stabilisation window. Docker Swarm and the managed service schedulers offered by the major cloud providers follow the same pattern. Van Do et al. (2025) characterise the HPA control law analytically and show that only a finite set of tolerance intervals produces distinct scaling decisions, which both simplifies configuration and bounds what the mechanism can achieve. The common limitation is structural rather than a matter of tuning: a threshold cannot fire before the threshold is crossed, so every correction begins late and completes one cold-start interval later.

B. Forecast-Driven Scaling

The dominant response has been to place a time-series model ahead of the scaling rule. Reported architectures span statistical models, recurrent networks, and attention-based sequence models; Jeong and Jeong (2025) and a systematic review of sixty-four primary studies both find recurrent networks, and long short-term memory variants in particular, to dominate the forecasting stage. Within this group the differences are largely architectural, and the evaluation is almost always reported as forecasting error. Fog et al. (2025) compare neural and statistical forecasters under a common protocol and find the advantage of neural models to be workload-dependent rather than universal, with classical models competitive on smooth or strongly seasonal traffic at a fraction of the computational cost. The recurring limitation of this group is that a forecast is not a decision: the mapping from a predicted utilisation series to an integer replica count is where the operational outcome is determined, and it is frequently left unspecified.

C. Optimisation-Based and Control-Theoretic Scaling

A smaller body of work treats scaling as a control problem. Incerto et al. (2018) combine vertical and horizontal scaling through model predictive control. Quattrocchi et al. (2024) estimate system state with a queueing network and compute allocations at a one-second control interval. Zou et al. (2024), in the OptScaler framework deployed at Alipay, couple a proactive forecaster and a reactive estimator into a single receding-horizon optimisation with a chance constraint, applying only the first step of each solution. Cai et al. (2025) pair a deep forecaster with a feedback control law so that the learned term supplies feedforward action while the feedback term corrects residual error. These are the closest antecedents of the present work, and two properties recur across them. Their decision variables are real-valued and the solution is rounded before use; and none reports a stability analysis, an oscillation metric, or a penalty on changing the decision between intervals.

D. Actuation Delay as an Explicit Quantity

Treating start-up latency as a first-class quantity is recent. Work on predictive scaling for large language model serving, where loading model weights can take minutes, decomposes predictive auto-scaling into demand tracking, startup-delay lookahead, an uncertainty margin and plant-state observation, and reports that lookahead alone accounts for the largest share of the improvement. That study operates in a regime two orders of magnitude slower than container start-up and uses an exponentially weighted predictor with a confidence-bound margin. Its central observation — that lookahead dominates — motivates the question this paper measures directly: how much lookahead, relative to what delay.

E. Gap Addressed Here

Across these groups, the integrality of the scaling decision is treated as a rounding detail and the relationship between horizon and actuation delay is treated as a configuration choice. Neither has been measured. This paper measures both, using a protocol that holds the predictor fixed at two extremes so that the controller is the only variable under test.

F. Comparative Summary

Table I consolidates the reviewed work. The grouping is by the mechanism each study uses to decide the replica count, because that is the axis along which the present contribution is positioned. Two columns deserve attention. The methodology column records whether the decision variable is discrete; across the optimisation group it is not. The limitation column records whether actuation delay and decision churn are treated as first-class quantities; across all groups they are not.

Ref.	Mechanism	Methodology	Dataset / benchmark	Reported outcome	Limitation
[1]	Threshold	HPA control law, analytic characterisation	Session-based CNF traffic	Finite distinct tolerance intervals; bounds on reactive scaling	Characterises the baseline only; no predictive mechanism
[2]	Forecast-driven	ETimeMixer with convolution; burst/non-burst state classification	Seven public cloud workload traces	Reports ≈ 70 % utilisation with fewer scaling updates	Scaling rule is a state-dependent heuristic, not an optimisation
[3]	Forecast-driven	Controlled comparison of statistical and neural forecasters	Kubernetes workload traces	Neural advantage is workload-dependent, not universal	Comparison study; contributes no scaling mechanism
[4]	Optimisation	MPC over combined vertical and horizontal scaling	Containerised web applications	First MPC formulation for container elasticity	Real-valued decisions; no dead-time term; no churn metric
[5]	Optimisation	Queueing-network state estimation, 1 s control interval	Two applications, seven workloads	Near-zero SLA violations at sub-millisecond decision cost	Reactive state estimation; no lookahead across actuation delay
[6]	Optimisation	Proactive forecaster + reactive estimator in one chance-constrained MPC	Alipay production traces, three clusters	Reports > 36 % fewer SLO violations than a hybrid baseline	Real-valued decisions, rounded for use; no integrality constraint; no churn penalty; 30 min intervals
[7]	Hybrid learning + control	Deep forecaster supplying feedforward, feedback law correcting residual	Containerised microservice benchmarks	Feedback term stabilises the loop when the predictor degrades	Controller gains tuned per application; no hard constraints
[8]	Forecast-driven	Convolution-augmented Transformer, local and global workload structure	Cloud-native and serverless traces	Captures short bursts and long periodicity jointly	Model footprint impractical for sidecar deployment
[9]	Graph / RL	Hierarchical graph neural network over container dependencies	Real-world microservice traces	Captures inter-service coupling in the scaling decision	No safety guarantee during exploration; sample-inefficient
[10]	Survey	Four-dimensional taxonomy over triggers, targets, models, metrics	Systematic review	Maps the design space; identifies drift-awareness gap	No empirical evaluation of the proposed indices

Table I. Reviewed work grouped by decision mechanism. Outcomes are as reported by the original authors and were not reproduced here; the present work reproduces none of these systems and makes no numerical comparison against their published figures.

Nowhere in the table is the replica count treated as an integer decision variable. The optimisation group, the closest antecedent of this work, relaxes integrality and rounds the solution, and none of those studies reports what the rounding costs. Actuation delay is handled explicitly in a single entry, and there at a granularity two orders of magnitude slower than container start-up. Decision churn is reported nowhere at all, although every scaling action consumes a container start or stop and redistributes live connections. Those three omissions are what Section V measures.

III. CONTROLLER DESIGN

The controller observes the pool once per interval Δt, converts a demand forecast into an integer replica requirement, and solves a constrained receding-horizon problem whose decision variable is the change in replica count. Fig. 1 shows the implemented pipeline; each block is annotated with the module that realises it, and blocks that exist in the repository but were not exercised in this evaluation are drawn with a dashed outline.

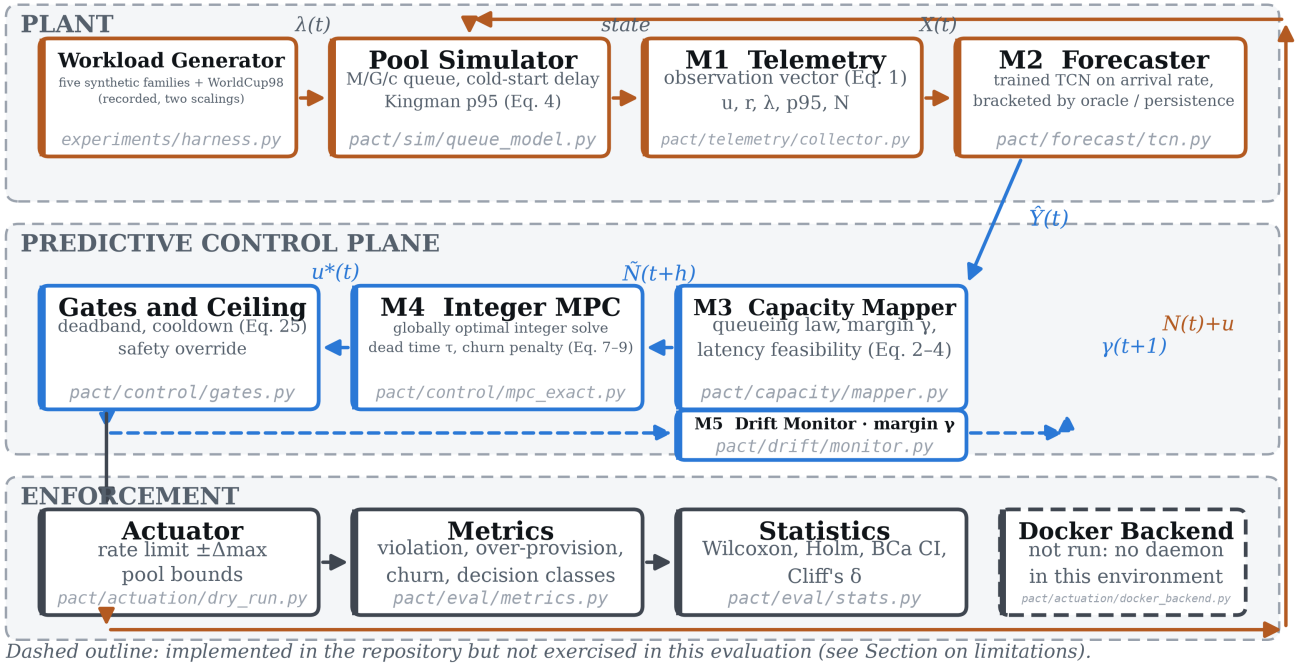


Fig. 1. Implemented pipeline. Each block names the repository module that realises it. The dashed outline marks the one component that exists in the repository but was not exercised in this evaluation: the Docker actuation backend (no daemon available). The forecaster block is the trained TCN on recorded traffic and the two bracketing predictors everywhere. Bold labels are component names.

A. *Observation and Capacity Mapping*

At each tick the collector assembles the observation vector of (1), where u is mean CPU utilisation across the pool, r is the resident memory fraction, λ the request arrival rate, ℓ the 95th-percentile response time, and N the current replica count.

$$m(t) = [\, u(t),\ r(t),\ \lambda(t),\ \ell(t),\ N(t)\,]^{T} \quad (1)$$

A forecast of utilisation is converted to a replica requirement through the utilisation law (2), in which u* is the target operating point, μ the per-replica service rate, and γ a safety margin. The ceiling is used because a fractional shortfall is a real shortfall.

$$\tilde{N}(t+h) \;=\; \left\lceil\, (1+\gamma)\cdot \frac{\hat{\lambda}(t+h)}{\mu\, u^{*}} \,\right\rceil \quad (2)$$

Memory imposes an independent bound, so the binding requirement is the larger of the two, as in (3).

$$\tilde{N}(t+h) \;\leftarrow\; \max\left\{\, \tilde{N}_{\mathrm{CPU}}(t+h),\ \left\lceil \hat{r}(t+h)\, N(t)/r^{*} \right\rceil \,\right\} \quad (3)$$

Expected tail latency at a candidate replica count is estimated by the heavy-traffic approximation (4), where $\bar{s}$ is the mean service time and c_a^2 and c_s^2 are the squared coefficients of variation of the arrival and service processes. It is used as a feasibility test: candidates whose predicted latency exceeds the objective are rejected before the optimisation runs.

$$\hat{\ell} \;=\; \bar{s} \;+\; \frac{c_a^2 + c_s^2}{2}\,\frac{\rho}{1-\rho}\,\bar{s}, \qquad \rho \;=\; \hat{\lambda}/(N\mu) \quad (4)$$

B. *Plant Model and Dead Time*

The pool evolves according to (5): the replica count changes by the control input applied τ ticks earlier, where τ is the cold-start interval expressed in control ticks. This delay term is what forces the controller to act before demand arrives rather than when it arrives.

$$N(t+1) = N(t) + u(t-\tau), \qquad \tau = \lceil \tau_c/\Delta t \rceil, \qquad u \in \mathbb{Z} \quad (5)$$

For the forecast to be actionable it must reach at least τ ticks into the future, which gives the feasibility condition (6). Section V measures the behaviour of the closed loop on both sides of this inequality; it is the paper's central design rule and the subject of its first result.

$$H\,\Delta t \;\geq\; \tau_c + \Delta t_{\mathrm{ctrl}} \quad (6)$$

C. *Receding-Horizon Objective*

The applied action minimises (7) over the horizon. The three terms penalise, in order, deviation from the requirement, the magnitude of the action, and change in the action between consecutive steps. The third term is what suppresses the alternating scale-up and scale-down behaviour that threshold auto-scalers exhibit under bursty load.

$$J(u) = \sum_{h=1}^{H} \left[\, q_h \big(N(t+h) - \tilde{N}(t+h)\big)^2 + r\,u(t+h-1)^2 + s\big(u(t+h-1) - u(t+h-2)\big)^2 \,\right] \quad (7)$$

The tracking weight is asymmetric, as in (8): being short of the requirement is penalised more heavily than being over it, because a dropped request costs more than an idle replica-second.

$$q_h = q^+ \ \text{ if } \ N(t+h) < \tilde{N}(t+h), \ \text{ else } \ q^-, \qquad q^+ \gg q^- \tag{8}$$

The optimisation is subject to (9): the pool is bounded, the per-tick change is rate limited, and the decision is an integer. The last of these is the constraint that related work relaxes.

$$N_{\min} \le N(t+h) \le N_{\max}, \quad |u| \le \Delta_{\max}, \quad u \in \mathbb{Z} \tag{9}$$

The integer constraint is kept rather than relaxed. Section III-E shows that the actuation dead time confines the decisions that can affect the horizon to its first few steps and renders the remainder separable, which is what makes an exact solve affordable here. Section V reports the cost of this choice in both directions: the solve time, which stays below 34.3 ms per tick, and the decisions it changes.

Only the first element of the optimal sequence is applied and the problem is re-solved at the next tick. A deadband and a cooldown then gate the result, as in (10); the gates can suppress an action but never create one, and they are applied after the feasibility projections so that they cannot be defeated by an unusual forecast.

$$u_{\text{applied}}(t) = u^*(t) \ \text{ if } \ |u^*(t)| \ge \theta \ \text{ and } \ t - t_{\text{up}} \ge T_{\text{cool}}, \ \text{ else } \ 0 \tag{10}$$

D. Margin Adaptation and Safety Ceiling

A drift monitor compares the replica count the observed load actually required against the count predicted τ ticks earlier and adapts the margin by the clipped integral update (11). A separate ceiling bypasses the optimiser and every gate when observed utilisation exceeds an emergency threshold, so that an unprecedented burst is not held back by the deadband. Sections V-E, V-F and V-Q report when each of them acted.

$$\gamma(t+1) = \text{clip}(\gamma(t) + \kappa_\gamma \, \bar{e}(t), \ \gamma_{\min}, \ \gamma_{\max}) \tag{11}$$

E. Algorithms

Algorithm 1 states the per-tick control loop. Lines 2 to 4 realise the observation and capacity mapping of Section III-A, line 6 the safety ceiling, line 9 the integer optimisation, and lines 11 to 14 the feasibility projections and gates in the order given in Section III-C. The order is load-bearing: projections precede gates, so an unusual forecast cannot defeat the pool bounds or the rate limit by routing around them.

Algorithm 1: PACT control tick

```
 1  input  N(t), pending, u_prev, γ(t), config
 2  obs   ← observe_pool()                  // Eq. (1)
 3  Ŷ     ← forecast(obs, H)                // predictor
 4  Ñ[1..H] ← capacity_map(Ŷ, N(t), γ)      // Eq. (2)-(4)
 5
 6  if obs.u > u_emergency then             // ceiling
 7     return immediate_scale_up(N(t), obs.u)
 8
 9  u* ← argmin J(u) over u ∈ [−Δmax, Δmax] ∩ ℤ  // Eq. (7)-(9)
10
11  u ← clip(u*, −Δmax, Δmax)               // rate limit
12  u ← clip(N(t)+u, Nmin, Nmax) − N(t)     // pool bounds
13  if |u| < θ then u ← 0                   // deadband
14  if u < 0 and t − t_up < T_cool then u ← 0   // cooldown
15
16  actuate(N(t) + u)
17  γ(t+1) ← margin_update(γ(t), e(t))      // Eq. (11)
18  return u
```

Algorithm 1. One control tick. Gates (13–14) can only suppress an action, never create one.

Algorithm 2 states the integer solve of line 9. Solving it exactly looks expensive: the decision space has $(2\Delta\max + 1)^H$ points, which is 9^{12} at the configuration used here. Two properties of the dead-time plant reduce it to something that can be enumerated.

The first is truncation. Because u(t+j) does not appear in N until tick t + j + τ, it can enter a tracking term $(N(t+k+1) - \tilde{N}(t+1+k))^2$ only when j + τ ≤ k + 1. Over k = 0 … H − 1 that confines the tracking-relevant actions to j ≤ H − τ, so only m = max(1, H − τ + 1) of the H decision variables can move the tracking cost at all. Saturation of N at the pool bounds does not weaken the argument, because saturation depends only on what has already landed. At τ = 9 and H = 12 this gives m = 4, an enumeration over $9^4 = 6561$ sequences rather than 9^{12}.

The second is separability of the remainder. Actions u(t+m) … u(t+H−1) land after the horizon ends, so they contribute only the decoupled chain $\Sigma[r\,u^2 + s(u - u_prev)^2]$ and their own admissibility. That is a scalar shortest-path problem in (committed, u_prev), minimised exactly by backward recursion. The remainder is not identically zero: with r = 0.15, s = 0.60 and u(t+m−1) = 4, holding the tail at zero costs 9.60 against 4.35 for a ramp-down. A completion that zeroes the tail therefore overstates the cost of every plan ending on a large action, and because the overstatement depends on the plan it does not cancel in the argmin.

Memoising the recursion on the full controllable state (k, N, pending, u_prev) keeps the procedure tractable at the opposite extreme. When τ is small the pending queue is short, so distinct action prefixes collapse onto the same state and the recursion is polynomial; when τ is large the queue is long but m is small and the tree is shallow. The two regimes are complementary, which is why a single procedure covers the whole grid swept in Section V-A.

Algorithm 2: Globally optimal integer first action

```
input  N, Ñ[1..H], pending, u_prev, config
m ← max(1, H − τ + 1)                    // truncation
memo ← { }

function V(k, n, p, prev):               // cost-to-go
  if k = m then
    return tail_track(n, p, Ñ) + chain(n, p, prev)
  if (k, n, p, prev) ∈ memo then return memo[·]
  v ← ∞;  cm ← committed(n, p)
  for a ∈ {−Δmax, ..., +Δmax} do
    if cm + a ∉ [Nmin, Nmax] then continue
    n′, p′ ← plant(n, p, a, τ)           // Eq. (5)
    q ← (n′ < Ñ[k+1]) ? q⁺ : q⁻           // Eq. (8)
    c ← q(n′ − Ñ[k+1])² + r a² + s(a − prev)²
    v ← min(v, c + V(k+1, n′, p′, a))
  memo[(k, n, p, prev)] ← v;  return v

best ← ∞;  u_best ← 0;  cm ← committed(N, pending)
for u ∈ {−Δmax, ..., 0, ..., +Δmax} do
  if cm + u ∉ [Nmin, Nmax] then continue
  n′, p′ ← plant(N, pending, u, τ)
  q ← (n′ < Ñ[1]) ? q⁺ : q⁻
  c ← q(n′ − Ñ[1])² + r u² + s(u − u_prev)²
  if c + V(1, n′, p′, u) < best then
    best ← c + V(1, n′, p′, u);  u_best ← u
return u_best
```

Algorithm 2. Exact integer solve. tail_track evaluates the tracking terms of stages m … H−1, which the truncation argument fixes independently of the remaining actions; chain is the backward recursion over the decoupled actuation and churn terms of those stages. Both are exact, so the returned first action is the global optimum.

The cost is $O((2\Delta\max + 1)^m \cdot H)$ when the dead time is long and polynomial in H when it is short, with no iteration to convergence and therefore no data-dependent worst case. Section V-H reports the measured time.

Correctness was checked against exhaustive enumeration rather than argued. Across ten configurations small enough to enumerate the full $(2\Delta\max + 1)^H$ decision space, the first action returned by Algorithm 2 was compared with brute force on 400 randomly drawn plant states; it attained the brute-force optimum in 400 of them. The same comparison prices the alternative of completing the horizon greedily: a solver that enumerates first actions and then tracks demand myopically returned an optimal first action in only 354 of those 400 cases. Section V-I quantifies the gap at the configuration used throughout this paper.

F. Module-Level Implementation

The controller is implemented as five modules with narrow interfaces, so that each can be disabled independently for the ablation of Section V-E and so that the measurement harness can substitute a predictor without touching the control path. This subsection records what each module does in the implementation, at the level of detail needed to reproduce it.

Telemetry and feature construction

The collector exposes a single method returning the observation vector of (1). Container runtimes report cumulative counters rather than rates: processor time arrives as consumed nanoseconds and network and block activity as byte totals. Each is differenced against the previous sample and divided by the interval before it carries meaning as load. Counter resets, which occur when a container restarts, would otherwise produce a large negative rate; the implementation detects a decrease and treats the new value as the first sample of a fresh counter rather than differencing across the discontinuity.

Each channel is then smoothed by a first-order exponentially weighted filter and mapped to the unit interval using per-channel bounds estimated once and frozen. Freezing matters: if the normalisation adapted online, the same raw utilisation would map to different model inputs at different times and the predictor would be chasing a moving target. The implementation enforces this by refusing to refit the normaliser after it has been frozen, which converts a subtle statistical error into an immediate exception.

Two derived channels are appended. The first is the per-replica arrival intensity, the arrival rate divided by the current replica count. Without it the predictor would be forecasting a signal that the controller itself modifies, because utilisation falls whenever the controller adds capacity; dividing it out leaves the exogenous demand. The second is the time of day encoded as a sine and cosine pair, which supplies daily periodicity without requiring the model to span a full day of history.

Capacity mapping

The mapper is a pure function with no internal state, which makes it trivially testable and means the ablation can replace it without disturbing anything else. For each horizon step it computes the processor-bound requirement from (2), the memory-bound requirement, takes the larger of the two as in (3), and then increases the candidate while the latency estimate of (4) exceeds the objective. That loop terminates because the candidate increases strictly and is bounded above by the pool maximum, a property worth stating because an unbounded search inside a control loop is a liveness hazard rather than merely a performance one.

The ceiling in (2) is deliberate. Rounding to nearest would accept a shortfall of up to half a replica as adequate, and at the small pool sizes this work targets half a replica can be several per cent of total capacity. The margin γ is applied before the ceiling rather than after, so that a small margin can still change the integer outcome rather than being rounded away.

Gate ordering

The gates apply in a fixed order: round the relaxed value, clip to the rate limit, clip to the pool bounds, apply the deadband, then

apply the cooldown. The order separates feasibility from policy. The first three steps guarantee the action is admissible with respect to the hard constraints of (9); the last two decide whether an admissible action is worth taking. Reversing them would allow a deadband or cooldown decision to be taken on an action that the bounds were about to modify, so that the gate would be reasoning about a value never applied.

The implementation records the intermediate value at each stage rather than only the final action. This is not instrumentation for its own sake: when a controller produces an unexpected replica count, the question is always which stage changed the decision, and a single logged number cannot answer it.

Drift monitoring

The monitor compares the replica count the observed load actually required at tick t against the count that was predicted for t by the forecast issued at $t - \tau$. Implementing that comparison requires retaining each issued horizon keyed by both its target tick and its issue tick, because the relevant prediction is a specific element of a specific past forecast rather than the most recent one. An off-by-one here is silent: the margin still adapts, it simply adapts against the wrong comparison, and nothing in the output distinguishes the two. The implementation therefore keys the buffer explicitly and the test suite verifies the alignment against hand-computed values.

The update itself is the clipped integral of (11). Clipping is what prevents a single anomalous period from driving the margin to a value that is expensive to unwind, and it is also what makes the mechanism deterministic in the sense this paper uses: the margin is a scalar with known bounds and a known update rule, inspectable in a log line, rather than a posterior that must be sampled.

Actuation

The simulated actuator applies the action to the plant with the same bounds and rate limit the controller assumed, which closes the loop consistently. The repository also contains a Docker backend that issues scale commands and reconciles the observed container count against the desired one, polling until they agree or a timeout expires. That backend is idempotent, so issuing the current count is a no-op, and it is asynchronous, so reconciliation never blocks a control tick. It was not exercised in this evaluation because no container runtime was available; Fig. 1 marks it accordingly.

IV. EXPERIMENTAL SETUP

A. Plant and Environment

Experiments run against the discrete-event pool simulator in the project repository. The simulator maintains a set of ready replicas and a set of pending replicas; a replica requested at tick t becomes ready at tick $t + \tau$ and contributes no capacity before then, so the cold-start interval is modelled explicitly rather than absorbed into a margin. Tail latency is computed from the heavy-traffic approximation of (4). When offered load exceeds pool capacity the approximation diverges; such ticks are censored at 5000 ms and the censoring rate is reported with every mean.

Environment. All runs executed on macOS-26.6.2-arm64-arm-64bit, Python 3.11.14, NumPy 2.4.6, PyTorch 2.14.0 and statsmodels 0.15.0, at repository commit 59f4ec0523 with uncommitted changes. The neural forecasters were trained on the MPS device of that machine and evaluated on its CPU. Each run is 400 control ticks at $\Delta t = 5$ s, giving a simulated duration of 33 minutes, and starts with four replicas or the requirement (2) of its first tick, whichever is larger. Results are reported over 10 seeds unless stated otherwise.

B. Plant Model

The simulated plant is the component that determines how far these results transfer, so its assumptions are stated explicitly rather than left to the source.

The pool maintains two sets, ready replicas and pending replicas. An action that adds capacity places the new replicas in the pending set with a release tick τ ahead of the present; they contribute no service capacity until that tick arrives. An action that removes capacity takes effect immediately, which is the asymmetry a real container runtime exhibits, since stopping a process is fast and starting one is not. Offered load is divided among ready replicas only.

Service is modelled as a multi-server queue whose per-replica rate is constant. Tail latency follows the heavy-traffic approximation of (4) with a p95 obtained by assuming an exponential sojourn distribution, which is exact for a single-server Markovian queue and an approximation otherwise. Memory is modelled as a fixed per-replica baseline plus a term proportional to utilisation. The approximation diverges as utilisation approaches unity; Section IV-A describes the censoring applied there and every reported mean carries its censoring rate.

Three properties of real deployments are absent and each bounds the conclusions. Replica start-up is a fixed latency rather than a distribution, so the variance that image-pull contention and scheduler queuing introduce is not represented. Replicas are identical, so heterogeneous hosts and noisy neighbours are absent. Requests are not queued across the capacity change, so the transient when a new replica joins and the load balancer redistributes connections is not modelled. All three would make the plant harder to control, and the measured service-level figures should therefore be read as optimistic in absolute terms even though the comparisons between controllers, which share the plant, remain valid.

C. Datasets

Two kinds of arrival trace are used. Five synthetic families are generated by the harness in the repository, chosen to span distinct demand shapes rather than to sample one shape repeatedly. Each is parameterised by a seed that perturbs it with additive Gaussian noise, and the same seed produces the same trace for every method, so comparisons are paired. The recorded workload is the 1998 World Cup web-site access log [11], days 41–50, converted to an arrival rate at the control interval and rescaled at two levels so that it fits the simulated pool; Section V-Q describes the trace, the scaling and the chronological split. For a recorded family the seed selects one of the disjoint windows of the test split, so recorded runs are paired in the same way.

Trace family	Shape	Provenance	Ticks per run	Seeds
Diurnal	Sinusoidal base, period 300 ticks	Synthetic (this work)	400	10
Burst	Two Gaussian bursts on a flat base	Synthetic (this work)	400	10
Ramp	Linear rise, plateau, linear fall	Synthetic (this work)	400	10
Step	Single discontinuity at mid-run	Synthetic (this work)	400	10
Sawtooth	Repeating linear rise, period 90 ticks	Synthetic (this work)	400	10
WC98/trace	Recorded; whole trace peaks at 85 % of capacity	WorldCup98 [11]	400	10
WC98/test	Recorded; test split peaks at 85 % of capacity	WorldCup98 [11]	400	10

Table II. Workload families. The synthetic traces are generated by experiments/harness.py; the recorded ones are windows of the WorldCup98 test split, produced by experiments/make_real_traces.py from the archived logs. Seeds gives the count used in the main comparison; other experiments state their own. Every method sees the identical trace for a given family and seed.

D. Forecasters and Bracketing

The controller consumes a forecast of utilisation and memory over its horizon, and three predictors supply it. The oracle has perfect foresight of the arrival series and represents the limit of what forecasting can contribute. Persistence repeats the most recent observation across the horizon and represents the absence of a model. These two brackets drive every experiment, so that a conclusion holding under both can be attributed to the control law rather than to the predictor.

The third predictor is the temporal convolutional network of the forecasting module, trained on the arrival rate of the recorded trace and frozen before any closed-loop run (Section V-Q). Its 12-step arrival forecast is converted to utilisation and memory by the same capacity law that converts the oracle's, so the three predictors differ only in the arrival series they supply. The trained arm is used only on recorded traffic, where there is a training split to learn from, and not in the horizon sweeps, because the checkpoint's horizon is fixed. As a forecaster it is compared against LSTM, GRU, bidirectional LSTM and Transformer baselines of matched size, an ARIMA model selected by AIC, and persistence, all scored on the same test origins.

E. Methods Compared

Five decision mechanisms are compared. All receive identical telemetry, run at the same control interval, actuate through the same path, and are subject to the same pool bounds and rate limit; only the decision rule differs.

- Integer-exact MPC, the controller of Section III, solving (7)–(9) exactly over the admissible integer actions.
- Relaxed-and-rounded MPC, identical in objective and constraints except that the integrality constraint of (9) is dropped and the first element of the solution is rounded before application. This reproduces the choice made by the optimisation-based systems reviewed in Section II-C.
- A reactive threshold controller implementing the HPA control law with its tolerance band and scale-down stabilisation window.
- A static configuration fixed at the level required to serve the peak of the trace.

F. Metrics and Statistics

Service-level violation rate is the fraction of ticks whose tail latency exceeds the objective. Over-provisioning ratio is the mean positive excess of the replica count over the requirement. Actuation churn is the number of ticks on which the replica count changed. Cost is replica-seconds per thousand requests served. Comparisons against the proposed controller use the Wilcoxon signed-rank test on paired runs, with Holm correction across the family of comparisons and Cliff's delta as an effect size; confidence intervals are bias-corrected and accelerated bootstrap intervals over 4000 resamples. Significance is never claimed without the corrected p-value.

G. Configuration and Reproducibility

Table III lists every parameter that affects the reported results. Values marked as swept are varied in the experiment indicated; all others are held fixed throughout. The repository reproduces every result file and figure from this configuration with the command sequence given at the end of this subsection.

Symbol	Parameter	Value	Set by
Δt	Control interval	5 s	Fixed
H	Forecast horizon	12 ticks (60 s)	Swept in E5, E9
τ_c	Cold-start latency	45 s	Swept in E5, E12
μ	Per-replica service rate	40 req/s	Fixed
u^*	Target utilisation	0.65	Swept in E6
r^*	Target memory fraction	0.75	Fixed
ℓ_{max}	Latency objective	200 ms	Fixed
c_a^2, c_s^2	Squared CVs of arrival, service	1.0, 1.0	Fixed
Nmin, Nmax	Pool bounds	1, 16	Nmax swept in E3
Δ_{max}	Per-tick rate limit	4 replicas	Swept in E8, E9
q^+ / q^-	Tracking asymmetry	8 / 1	Swept in E8
r	Actuation weight	0.15	Fixed
s	Churn weight	0.60	Swept in E8
θ	Deadband	1 replica	Swept in E8
T_cool	Cooldown	60 s	Swept in E8
u_emergency	Safety-ceiling threshold	0.92	Fixed
κ_γ, η	Margin gain, error smoothing	0.02, 0.20	Fixed
γ_{min}, γ_{max}	Margin bounds	0.00, 0.50	Fixed
—	Ticks per run	400	Fixed
—	Seeds per configuration	10 (E1, E4); 3–8 elsewhere	Per experiment
—	Latency censoring ceiling	5000 ms	Fixed

Table III. Complete parameter set. Values are those used unless the experiment named in the final column varies them.

Reproduction. From a clean checkout: run_all.py writes the main comparison, the significance tests, the rounding gap, the ablation, the horizon sweep, the frontier and the corruption sweep; run_e8_part.py runs the control-parameter sweeps one at a time and merge_e8_parts.py assembles them; run_more.py writes the timing and breakdown files; run_trace.py writes the decision trace; verify_exact.py reproduces the solver validation of Section V-I. make_real_traces.py builds the WorldCup98 series; train_forecasters.py trains the TCN and the neural baselines; run_real.py writes E16–E22. make_figures.py, make_figures2.py, make_fig_exact.py, make_fig_real.py, make_arch.py, make_solver_compare.py and make_equations.py regenerate every plot, diagram and display equation; node paper.js rebuilds this document, reading each reported value from the result files at build time. Two scripts check the output rather than produce it: verify_numbers.py recomputes the headline quantities from the result files and fails unless each appears in the rendered document, and run_tests.py (or pytest) runs the test suite. The experiment stage takes hours rather than minutes, because the exact solve is about two orders of magnitude more work per tick than the relaxation it replaces. The stages that recorded their own duration total 6.5 hours on the platform named in results/manifest.json, and the 2 stages that predate that logging appear in results/runtime_stages.csv without a duration, so the figure is a lower bound. The same suite under the greedy-completion solver took about six minutes. That ratio is the price of the exactness claim, and it is paid offline: per tick the worst observed solve is 0.69 % of the control interval.

V. RESULTS AND DISCUSSION

All values in this section are computed from the run logs written to the repository's results directory. Every table and figure from here on is generated directly from those files.

A. The Horizon–Dead-Time Law

The first experiment sweeps the cold-start interval τ_c over six values and the forecast horizon H over ten, and measures the resulting violation rate for the proposed controller and for the reactive baseline. Fig. 2 shows the outcome as a heat map with the boundary $H\Delta t = \tau_c$ from (6) drawn on it, and Fig. 3 shows the same data as violation against lookahead for each cold-start setting.

The transition is sharp and it coincides with the predicted boundary. For every cold-start setting tested, violation rate is flat and elevated while the horizon is too short, then falls abruptly at the first horizon satisfying $H\Delta t \geq \tau_c$, and is flat and low thereafter. The grid covers six cold-start values, but at the shortest of them every horizon tested already satisfies the bound, so that setting yields no below-boundary measurement and is excluded from the average. Over the remaining 5, which bracket the boundary on both sides, the rate falls from 2.92 % below the boundary to 0.27 % above it. The effect is monotone in τ_c: at τ_c = 5 s the controller is already near its floor at the shortest horizon tested, whereas at τ_c = 90 s it remains at 5.42 % until the horizon reaches 20 ticks.

The comparison against the reactive baseline is the operationally important part, and it is not flattering to naive predictive scaling. When the horizon does not cover the dead time the predictive controller is not merely no better than reacting; it is worse, by up to 3.69 percentage points. Once the horizon covers the dead time it is better, by up to 3.58 points. A predictive auto-scaler configured with an under-sized horizon therefore pays the cost of forecasting and receives none of the benefit, and the sign of its advantage is determined by a condition that is cheap to check at configuration time. What makes the condition useful is that it can be checked before deployment: it compares the configured horizon against the measured ready-time of the image, and needs no model of the workload.

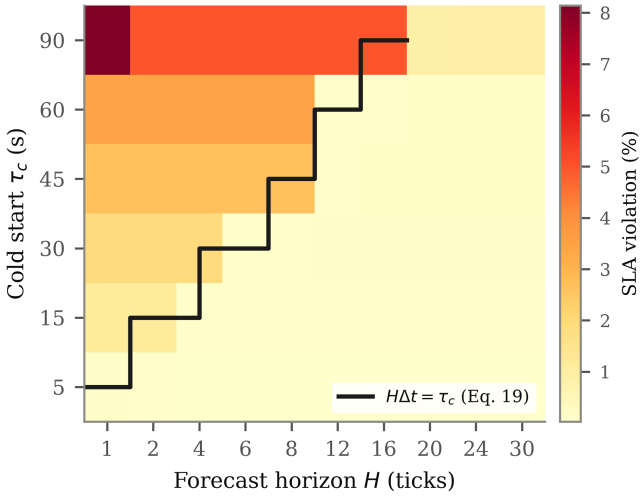


Fig. 2. Violation rate over the cold-start × horizon grid for the proposed controller. The solid line is the boundary HΔt = τc from (6). Synthetic traces; mean of three families × three seeds per cell.

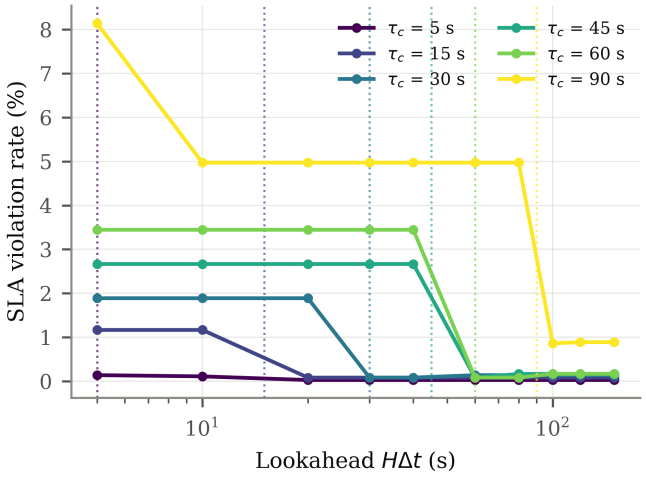


Fig. 3. The same data as violation against lookahead. Dotted verticals mark τc for each curve. Synthetic traces.

τc (s)	Ticks of delay ⌈τc/Δt⌉	First H meeting (6)	Violation below boundary (%)	Violation above boundary (%)	Reactive violation (%)	Worst margin below (pp)	Best margin above (pp)
5	1	1	n/a	0.05	0.17	n/a	0.14
15	3	4	1.17	0.08	0.42	-0.75	0.33
30	6	6	1.89	0.12	1.06	-0.83	0.97
45	9	12	2.67	0.15	1.75	-0.92	1.67
60	12	12	3.44	0.13	2.58	-0.86	2.50
90	18	20	5.42	0.88	4.44	-3.69	3.58

Table IV. The horizon–dead-time transition, by cold-start setting. Violation columns average over the horizons on each side of the boundary HΔt = τc. The margin columns give the difference from the reactive baseline, where a negative value means the predictive controller was worse. At τc = 5 s the shortest horizon tested already satisfies (6), so no value falls below the boundary. Cells whose plan exceeded the exact solver's node budget fell back to the greedy completion: 14 of the 60 cells, all with H ≥ 12, 14 of them above the boundary. Synthetic traces (burst, step and ramp), 9 runs per cell.

Reading the table by row makes the mechanism concrete. At τc = 45 s the plant needs 9 ticks of lookahead; the sweep tested H = 8 and H = 12 on either side, and the violation rate falls from 2.67 % to 0.15 % between them. The reactive baseline on the same traces sits at 1.75 %, so an under-configured predictive controller is 0.92 points worse than not predicting at all, while a correctly configured one is 1.67 points better. Of the 5 delays at which the sweep tested horizons on both sides of the boundary, 5 show this pattern, and the boundary moves with τc as (6) requires rather than at some fixed horizon that would indicate a different cause.

B. *The Cost of Relaxing Integrality*

The second experiment compares the integer-exact controller against the relaxed-and-rounded controller across five pool bounds, holding everything else fixed. Table V reports the result and Fig. 4 shows it graphically.

N_{max}	Ticks where decisions differ (%)	Churn, exact	Churn, rounded	Δ violation (pp)	Mean solve (ms)
2	18.8	46.4	9.1	2.12	0.089
4	24.7	43.2	28.6	5.36	0.859
8	33.3	49.6	48.0	1.98	6.366
16	42.5	45.2	100.4	0.50	12.435
32	63.6	53.3	202.0	0.80	17.542

Table V. Integer-exact against relaxed-and-rounded control, by pool bound. Five synthetic families and two recorded WorldCup98 scalings, 5 seeds each, pooled per row. A positive Δ violation means the rounded controller violated more often.

The two controllers disagree on a substantial and increasing fraction of ticks, from 18.8 % at the smallest pool to 63.6 % at the largest. Pooled over families, the rounded controller violates the service level more often at 5 of the 5 pool bounds, by between 0.50 and 5.36 percentage points; the largest gap is at N_{max} = 4, where one replica is a large share of the pool. Churn depends on the pool bound. At N_{max} = 16 the exact controller issues 45 scaling actions per run against 100 for the rounded controller, and at N_{max} = 32 the figures are 53 against 202. Rounding converts a fractional preference into a full replica change, and because the fractional part fluctuates from tick to tick the rounded controller oscillates where the exact controller holds. It does not hold at N_{max} = 2, 4 and 8, where the rounded controller issues fewer actions (9 against 46; 29 against 43; 48 against 50); this experiment does not isolate why, so the churn advantage is claimed only for the larger pools.

The cost of solving exactly is small. The mean solve time is at most 17.54 ms per tick, at the largest pool tested, which is two orders of magnitude inside the 5 s control interval. This experiment, the solve-time sweep of Section V-H and the solver comparison of Section V-I were timed with no other experiment running on the machine. On this evidence the integrality constraint should not be relaxed for performance reasons at these problem sizes.

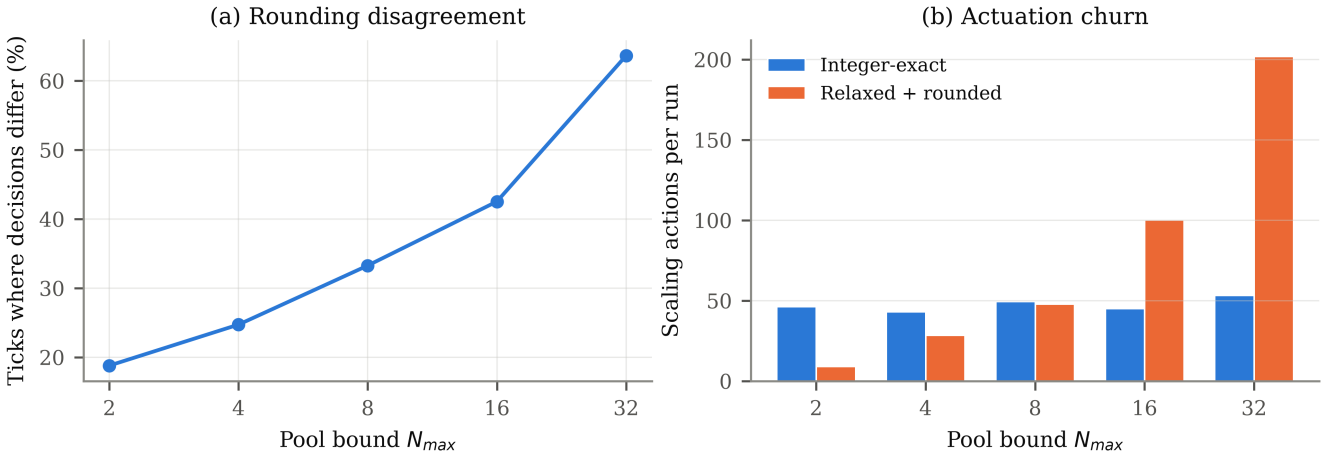


Fig. 4. Disagreement between the exact and rounded controllers, and the resulting actuation churn. Five synthetic families and two recorded WorldCup98 scalings.

C. *Operating-Point Sensitivity*

Comparing controllers at a single target utilisation is misleading, because each sits at a different point on the trade-off between service level and capacity. The third experiment therefore sweeps the target from 0.50 to 0.85 and traces the attainable frontier for each method. Table VI reports the extremes and Fig. 5 shows the full sweep.

Target u*	Exact MPC viol. (%)	Exact MPC over (%)	Rounded viol. (%)	Rounded over (%)	Reactive viol. (%)	Reactive over (%)
0.50	0.00	16.2	0.32	3.1	0.39	38.8
0.65	0.56	23.0	1.18	3.2	0.75	44.4
0.75	1.15	32.5	5.04	4.8	1.61	42.9
0.85	2.86	37.2	16.18	7.9	5.05	48.0

Table VI. Service level and capacity cost against target utilisation. Five synthetic families and two recorded WorldCup98 scalings, 21 runs per cell, oracle predictor.

The two controllers degrade very differently as the target is pushed up. The rounded controller is efficient and adequate at conservative targets — at u* = 0.50 it violates on 0.32 % of ticks while carrying only 3.1 % excess capacity — but it collapses as the target rises, reaching 16.18 % violations at u* = 0.85. The exact controller rises far more slowly over the same range, from 0.00 % to 2.86 %. The mechanism is visible in Table V: when the pool is run close to its limit, a rounding error of one replica is a large fraction of the available headroom, and the rounded controller spends an increasing share of its time on the wrong side of the requirement.

Neither controller dominates. The exact controller buys service level with capacity: at u* = 0.65 it carries 23.0 % excess against the rounded controller's 3.2 %, for a violation rate of 0.56 % against 1.18 %. An operator who can tolerate the violation rate of the rounded controller should prefer it on cost. The case for exact solution is that it remains usable at operating points where the rounded controller does not; its computational cost is reported in Section V-H.

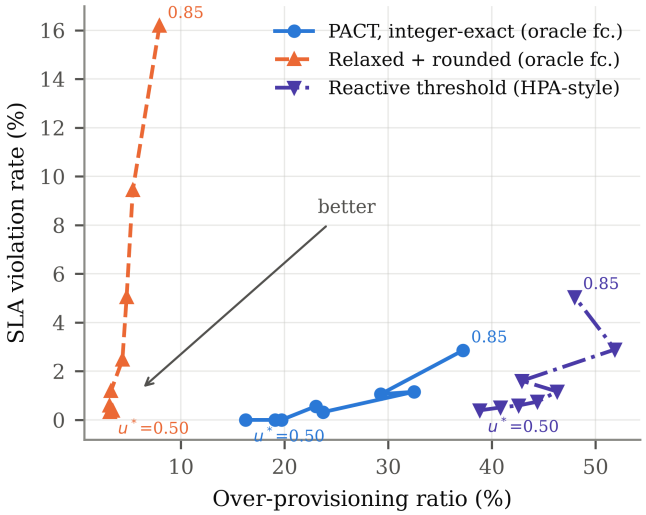


Fig. 5. Attainable frontier as the target utilisation is swept from 0.50 to 0.85. Five synthetic families and two recorded WorldCup98 scalings, oracle predictor. Points are labelled with the target that produced them.

D. *Aggregate Comparison and Significance*

Table VII reports the main comparison over 7 workload families × 10 seeds, with bootstrap confidence intervals, under both forecaster brackets. Table VIII reports the paired significance tests against the proposed controller with the oracle predictor.

Method	Predictor	Violation (%) mean [95% CI]	Over-prov. (%) mean [95% CI]	Churn	Util. (%)	Cost /1k req
Integer-exact MPC	Oracle	0.61 [0.33, 1.06]	23.6 [20.1, 27.6]	44.4	54.7	51.2
Integer-exact MPC	Persistence	0.75 [0.49, 1.07]	29.0 [26.8, 31.3]	57.4	53.4	54.1
Relaxed + rounded	Oracle	1.21 [0.83, 1.74]	3.2 [3.0, 3.5]	100.8	64.6	43.0
Relaxed + rounded	Persistence	1.16 [0.79, 1.66]	3.9 [3.4, 4.4]	102.2	62.7	44.7
Reactive threshold	—	0.71 [0.46, 1.01]	39.2 [32.2, 46.6]	12.1	48.9	58.0
Static over-provisioned	—	0.00 [0.00, 0.00]	70.3 [59.7, 82.0]	0.6	40.7	65.6

Table VII. Main comparison. Five synthetic families and two recorded WorldCup98 scalings, 10 seeds each = 70 runs per row. Intervals are 95 % bias-corrected bootstrap over 4000 resamples.

Comparison	Metric	Proposed	Other	p (Holm)	Cliff's δ	Significant
Relaxed + rounded (oracle)	Violation	0.61	1.21	0.103	-0.25	no
Relaxed + rounded (oracle)	Over-provisioning	23.59	3.25	< 0.001	1.00	yes
Relaxed + rounded (oracle)	Churn	44.39	100.84	< 0.001	-0.88	yes
Reactive threshold	Violation	0.61	0.71	1.000	-0.04	no
Reactive threshold	Over-provisioning	23.59	39.22	< 0.001	-0.27	yes
Reactive threshold	Churn	44.39	12.09	< 0.001	0.98	yes
Static over-provisioned	Violation	0.61	0.00	< 0.001	0.27	yes
Static over-provisioned	Over-provisioning	23.59	70.31	< 0.001	-0.66	yes

Table VIII. Paired Wilcoxon signed-rank tests against the integer-exact controller with the oracle predictor, Holm-corrected across the comparison family. n = 70 paired runs. Five synthetic families and two recorded WorldCup98 scalings.

Two aspects of Table VIII deserve emphasis because they qualify the paper's claims. First, the difference in violation rate between the proposed controller and the rounded controller at this operating point is not significant after correction ($p = 0.103$), and the difference against the reactive baseline is not significant either ($p = 1.000$). The service-level advantage reported in Section V-C emerges at aggressive operating points, not at the default one. Second, the differences that are significant are in churn and capacity: against the rounded controller the reduction in churn has a Cliff's delta of -0.88, and against the reactive baseline the reduction in over-provisioning has a delta of -0.27. The robust claims are about stability and cost, not about service level in the ordinary regime.

The forecaster brackets bound the predictor's contribution under this protocol. Moving from perfect foresight to persistence changes violations from 0.61 % to 0.75 % and churn from 44 to 57 actions for the exact controller, an interval of 0.14 percentage points in violation rate, and from 1.21 % to 1.16 % for the rounded one. The ordering of the exact and rounded controllers is the same under both brackets for 3 of the 3 metrics compared (violation, over-provisioning, churn), which is the condition under which a difference is attributed to the control law. Section V-Q tests the further assumption that a trained forecaster lands inside the bracket, and finds that it often does not.

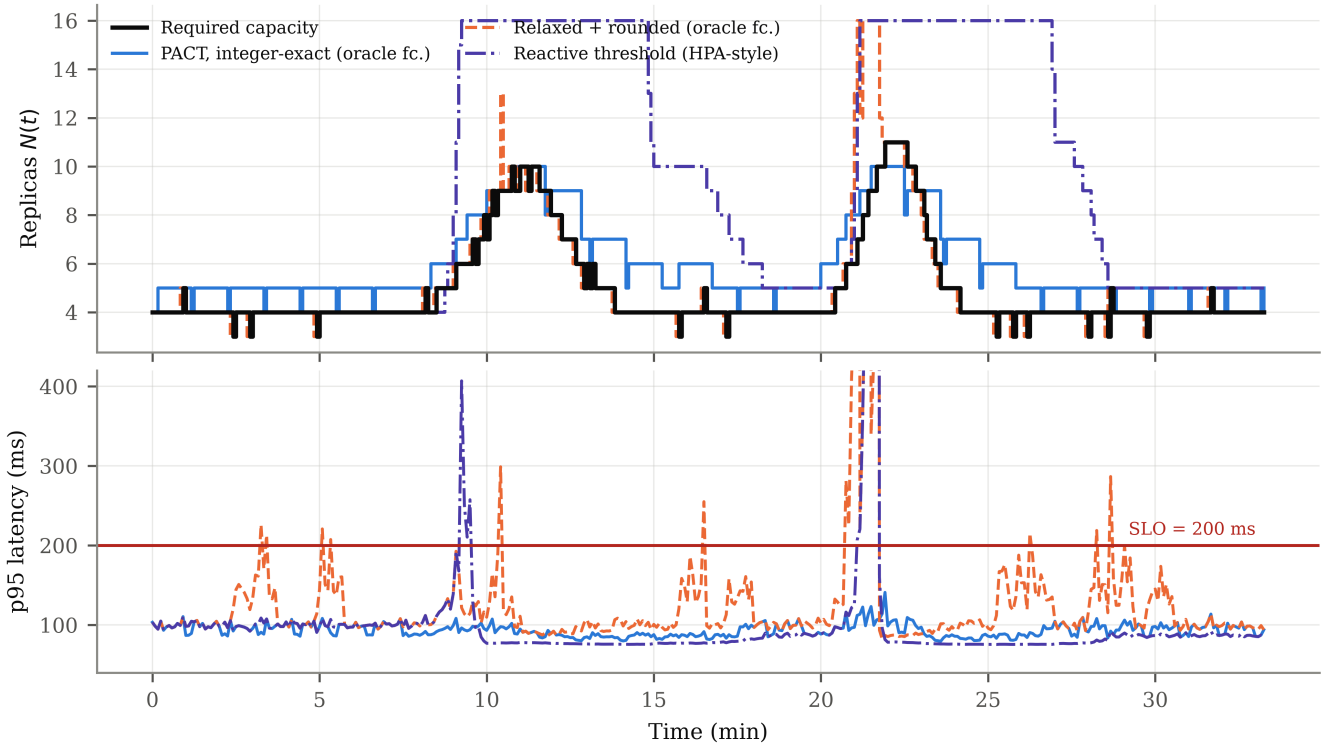


Fig. 6. Replica count and tail latency on one burst trace. The rounded controller overshoots at each burst and the reactive controller holds capacity long after demand falls. Synthetic burst trace, seed 1001, oracle predictor.

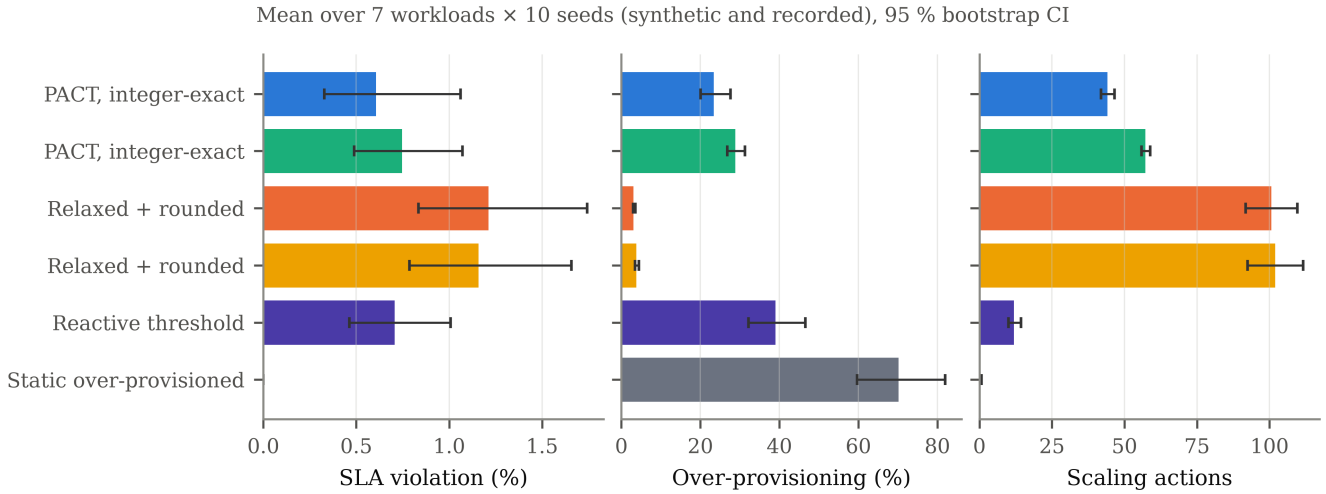


Fig. 7. Main comparison with 95 % bootstrap intervals. Five synthetic families and two recorded WorldCup98 scalings, 70 runs per bar.

E. Ablation

Each component of the controller was disabled in turn, with everything else held fixed, over five synthetic families and two recorded WorldCup98 scalings, 10 seeds each, with the oracle predictor. Table IX reports the outcome and Fig. 8 shows it. No component changes the violation rate significantly: 5 of the 5 service-level comparisons have $p \geq 0.05$ uncorrected. What the components change is how much capacity the controller holds and how often it moves.

Removing the optimisation layer entirely, so that the controller applies the first element of the demand trajectory directly, changes the violation rate by 0.20 percentage points ($p = 0.411$) and reduces over-provisioning by 16.4 points. Removing the churn penalty changes service level by -0.15 points but raises over-provisioning by 14.5 points and churn by 13 actions, consistent with the rounding result of Section V-B: penalising change is what keeps the pool steady.

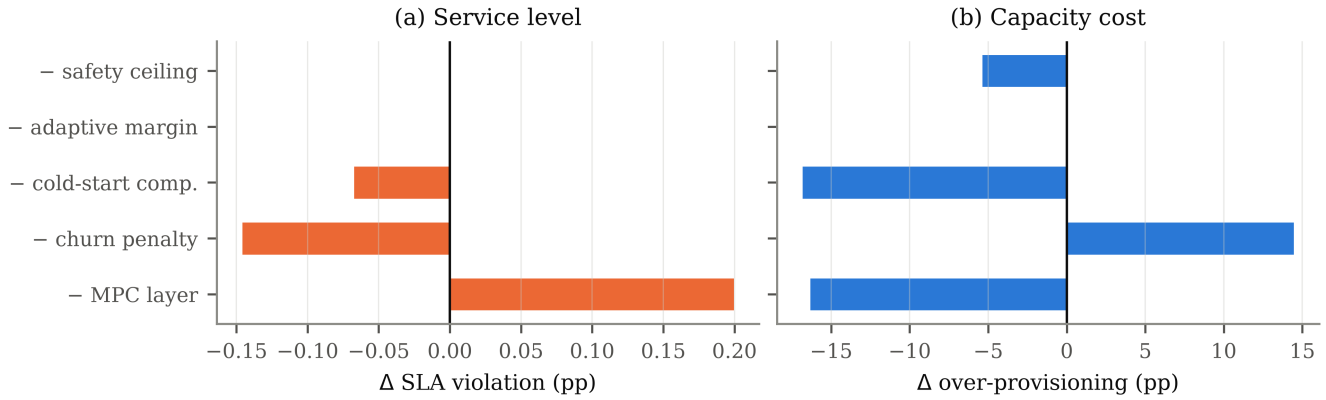
Disabling cold-start compensation, so that the controller plans as if a replica were ready the moment it is requested, changes the violation rate by only -0.07 points ($p = 0.700$), but it cuts over-provisioning by 16.9 points and churn by 19.9 actions. An earlier version of this paper reported only the first of these and called the component ineffective; the capacity and churn effects were present in that version's result files as well. The delay term makes the controller provision ahead of the dead time, and on these workloads, with a horizon that already covers the dead

time, the measurements do not show a service-level return for the capacity it holds. This does not contradict Section V-A, where the horizon itself was varied.

The two protective mechanisms behave differently. The adaptive margin moved in 0 of 50 synthetic runs and 3 of 20 recorded runs of the full controller, and disabling it changes violations by 0.00 points and over-provisioning by -0.003 points: under perfect foresight it is close to inert. It is not inert under persistence: in the main comparison it moved in 68 of 70 runs of the exact controller with the persistence predictor. The safety ceiling, by contrast, is active: it issued 160 overrides in 19 of 50 synthetic runs and 0 in the 20 recorded runs. Removing it leaves the violation rate exactly unchanged while lowering over-provisioning by 5.4 points and churn by 5.0 actions, so in these runs it bought capacity without a measured service-level return. An earlier version of this paper stated that the ceiling never activated; that was wrong for the main comparison and the ablation, and has been corrected.

Variant	Violation (%)	Δ vs. full (pp)	Over-prov. (%)	Δ vs. full (pp)	Churn	p (uncorr.)
Without MPC layer	0.89	0.20	6.5	-16.4	38.5	0.411
Without churn penalty	0.54	-0.15	37.4	14.5	57.2	0.705
Without cold-start comp.	0.62	-0.07	6.0	-16.9	24.3	0.700
Without adaptive margin	0.69	0.00	22.9	-0.0	44.6	1.000
Without safety ceiling	0.69	0.00	17.5	-5.4	39.2	1.000

Table IX. Ablation. Full controller: violation 0.69 %, over-provisioning 22.9 %, churn 44.2. Five synthetic families and two recorded WorldCup98 scalings, 70 runs per row, oracle predictor. No service-level delta reaches p < 0.05.



Negative is lower than the full controller; none of the service-level deltas reach $p < 0.05$.

Fig. 8. Effect of disabling each component. Five synthetic families and two recorded WorldCup98 scalings.

F. Robustness to Forecast Corruption

The final experiment injects multiplicative noise into the series seen by the forecaster while the plant continues to receive the true arrivals, sweeping the corruption magnitude and repeating the sweep with the safety ceiling disabled. Fig. 9 shows the result, which is counter-intuitive and worth stating carefully.

Violation rate does not rise with corruption; it falls slightly, from 0.083 % at zero noise to 0.000 % at σ = 0.4, while over-provisioning rises from 18.2 % to 64.5 %. The explanation is the asymmetric tracking weight of (8). Multiplicative noise causes the forecast to over-predict as often as it under-predicts, but the controller responds to over-prediction by adding capacity and to under-prediction by removing it, and the asymmetric penalty makes the former cheaper. Noise therefore acts as an accidental safety margin, purchased with capacity. This is a property of the cost function rather than a desirable robustness result, and it means the controller is not stress-tested by symmetric noise; a corruption model with sustained negative bias would be a sharper test and is left for future work.

The safety ceiling issued overrides at 7 of the 8 corruption levels, 14 in total, but the violation rate with and without it is identical at every level; only over-provisioning moves, by at most 0.92 points. The reactive baseline, measured on the same traces, violates on 1.63 % of ticks, above every point of the swept range. Symmetric noise does not produce the sustained under-prediction the ceiling exists to catch, so this experiment does not show what it is worth.

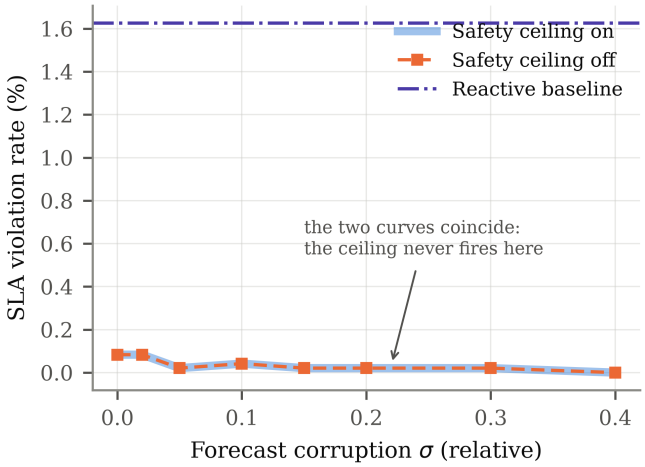


Fig. 9. Violation rate against forecast corruption, with and without the safety ceiling. Synthetic traces (burst, step and diurnal), 12 runs per point.

G. *Control-Parameter Sensitivity*

The controller exposes five parameters that shape the trade-off between service level and actuation stability. Each was swept independently over five synthetic families and two recorded WorldCup98 scalings, 8 seeds or windows each, holding the others at their defaults. Table X reports the sweeps and Fig. 10 shows them. The results reorder the components by importance relative to what the design assumed.

Parameter	Value	Violation (%)	Over-prov. (%)	Actions	Oscillations	Cost /1k
Churn weight s	0.00	0.50	36.9	57.5	52.6	56.3
	1.00	0.42	22.2	44.1	33.7	50.6
	2.00	0.38	21.8	25.8	15.5	50.5
Cooldown T_cool (s)	0	1.74	17.6	272.1	211.0	48.4
	60	0.45	22.0	44.5	35.2	50.6
	120	0.15	24.3	26.1	18.6	51.6
Deadband θ	1	0.45	22.0	44.5	35.2	50.6
	2	0.60	26.6	7.6	3.5	52.1
	3	1.38	31.4	5.7	1.6	54.0
Asymmetry q^+/q^-	1	0.85	20.2	39.5	29.5	49.6
	20	0.39	21.7	44.3	35.1	50.5
Rate limit Δmax	1	0.39	20.0	45.5	33.0	49.8
	8	0.45	22.0	44.3	35.2	50.6

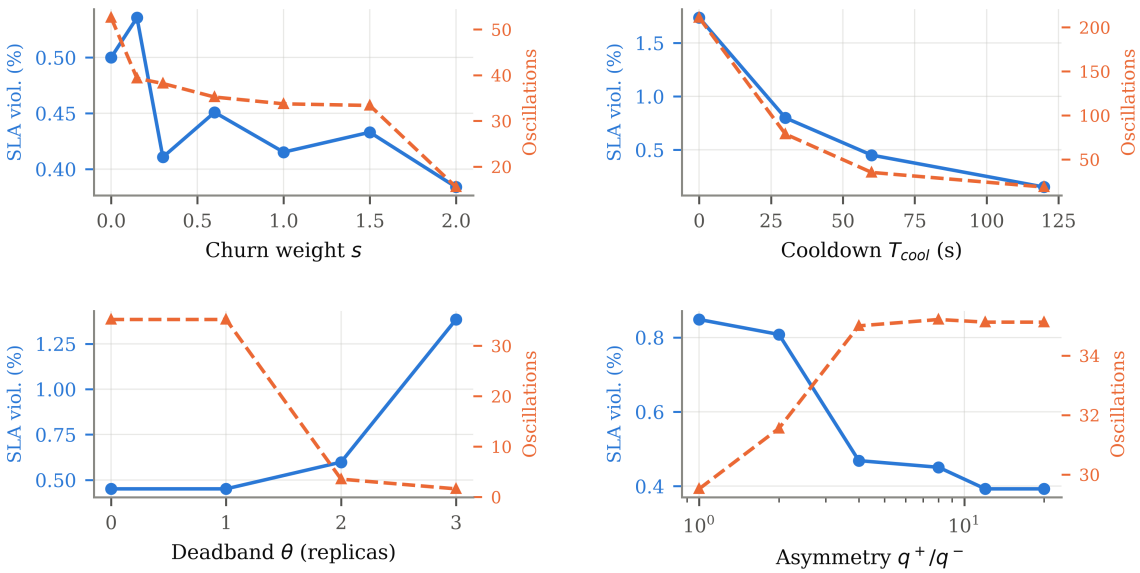
Table X. Control-parameter sweeps. Five synthetic families and two recorded WorldCup98 scalings, 56 runs per row; the remaining parameters are held at their defaults. Oscillations counts sign reversals in the replica delta.

The churn penalty is not a pure trade-off. Raising s from zero to unity changes violations from 0.50 % to 0.42 %, over-provisioning from 36.9 % to 22.2 %, and oscillations from 52.6 to 33.7, improving all three at once. Between s = 1 and s = 2 violations move to 0.38 % and over-provisioning to 21.8 %, while oscillations fall again, to 15.5, and actions to 25.8. Penalising change suppresses the scale-down-then-scale-up pairs that would otherwise remove capacity immediately before it is needed again, so the controller holds a steadier pool that is also a smaller one. This was not an anticipated result; the penalty was introduced to reduce churn alone.

The cooldown dominates stability, and by a wider margin than the churn penalty does. Removing it entirely raises the action count from 44.5 to 272.1 per run and oscillations from 35.2 to 211.0, while violations change from 0.45 % to 1.74 %. Extending it to 120 s gives 26.1 actions at 0.15 % violations. It is not free: over-provisioning rises from 17.6 % without a cooldown to 24.3 % at 120 s. Fig. 11 plots this sweep in the violation-versus-churn plane. An operator tuning this controller should set the cooldown first and the churn weight second.

The deadband is inert at its default. Values of θ = 1 and θ = 0 give identical results, because an integer action whose magnitude is below one is already zero; the deadband only becomes active at θ = 2, where it cuts actions from 44.5 to 7.6 at a cost of 0.15 percentage points of service level, and at θ = 3 it becomes harmful, reaching 1.38 % violations. The asymmetric tracking weight is not inert: raising q^+/q^- from 1 to 20 changes violations from 0.85 % to 0.39 % and over-provisioning from 20.2 % to 21.7 %.

The rate limit stops mattering above its default. Raising Δmax from 4 to 8 changes the violation rate by 0.00 points, over-provisioning by 0.03 points and the action count by -0.21. Pool bounds, not the rate limit, constrain the action set once Δmax is of the order of the pool itself: an action that would move more than a few replicas is already inadmissible because it would land outside [Nmin, Nmax]. The cost of raising it is not small. The exact search is O((2Δmax + 1)^m), and at the default horizon Table XI measures a mean solve time of 10.4 ms at Δmax = 4 and 56.2 ms at Δmax = 8, 5.4 times as much. Raising Δmax above four buys a controller that behaves almost the same and costs several times as much to compute.



Left axis (blue circles): service-level violation. Right axis (orange triangles): oscillation count. Synthetic and recorded traces, 7 families × 8 seeds per point.

Fig. 10. Control-parameter sweeps. Blue circles on the left axis are the violation rate; orange triangles on the right axis are the oscillation count. Five synthetic families and two recorded WorldCup98 scalings.

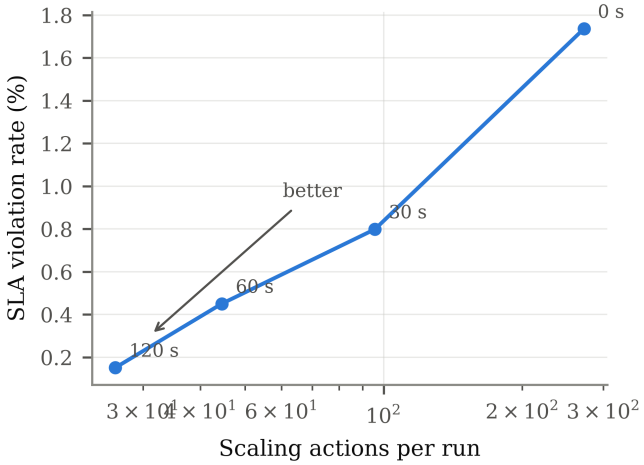


Fig. 11. The cooldown sweep in the violation-versus-churn plane. Points are labelled with the cooldown that produced them. Five synthetic families and two recorded WorldCup98 scalings.

H. *Computational Cost of Exact Solution*

If exact integer solution were expensive, relaxation would be justified regardless of the accuracy argument. Algorithm 2 was timed across the action-set size and the horizon, the two parameters that determine its work, with the dead time held at its default. The sweep answers two questions at once: what the exact solve costs where it is available, and where it stops being available at all. Table XI reports the extremes and Fig. 12 the full grid. Cells that exceeded the node budget were solved by the greedy completion instead and are marked as such; their timings describe that solver, not this one.

Δmax	Candidates	H	Solver	Mean solve (ms)	Max (ms)	Share of 5 s budget (%)
1	3	4	exact	0.0222	0.0243	0.0004
1	3	12	exact	0.3870	0.4194	0.0077
1	3	30	fallback	0.1555	0.1564	0.0031
4	9	4	exact	0.2058	0.2168	0.0041
4	9	12	exact	10.4180	10.8163	0.2084
4	9	30	fallback	0.4535	0.4620	0.0091
16	33	4	exact	2.0485	2.1478	0.0410
16	33	12	fallback	0.5183	0.5564	0.0104
16	33	30	fallback	1.2646	1.2771	0.0253

Table XI. Solve time against action-set size and horizon, at the default dead time. Rows marked “fallback” exceeded the exact solver’s node budget and were solved by the greedy completion, so their times are not times for Algorithm 2. Synthetic traces, 2 families × 4 seeds per row. The control interval is 5 s.

At the configuration used throughout the paper the mean solve time is 10.418 ms, which is 0.2084 % of the control interval. The costliest configuration the exact solver completed is Δmax = 8 at a 12-step horizon: 56.2 ms on average and 57.5 ms at worst. Larger cells in the grid are faster only because the exact search was abandoned there, so their timings belong to the greedy completion and are excluded here; Fig. 12(a) draws them hollow and unconnected for the same reason.

Fig. 12(b) plots those same exact solves against the number of prefixes the search enumerates. Two regimes are visible and they do not lie on one curve. Where m = 1 the enumeration is trivial and the cost is set by horizon length, because the recursion still walks every stage to evaluate the tail. Where m = 4 the enumeration dominates, and the measured cost grows markedly more slowly than the prefix count: from 81 prefixes to 83 521, a factor near a thousand, the mean solve rises from 0.8 ms to 109 ms, a factor of 140. Landing admissibility is the reason. The prefix count assumes every action in [−Δmax, Δmax] is available at every step, and most are not: an action that would carry the committed total outside the pool bounds is rejected before the search descends. The nominal count is an upper bound on the work, not an estimate of it.

The frontier matters as much as the timings. Exact solution is affordable in two regions — short horizons, where few actions reach inside the horizon at all, and horizons long enough relative to the dead time that the pending queue stops distinguishing states — but not in the band between them, and not at large action sets, where the branching factor dominates both. The default configuration sits inside the affordable region, which is why exact results can be reported throughout this paper; a pool with a much larger rate limit would not allow it.

The consequence for the paper's argument is narrower than it was before the solver was corrected. Relaxation saves real compute: at the default configuration it is the difference between 10.4 ms and a few tenths of a millisecond per tick. What it does not save is anything that is scarce. The saving is 0.21 % of a control interval that the controller spends idle, set against a behavioural difference on a large fraction of control decisions. Relaxation at container-pool scale is therefore not a performance optimisation but a modelling choice, and one whose cost is now measured. The argument does not carry to cluster-scale problems with hundreds of coupled decision variables: there the exact search is not available at all, as the frontier above already shows in miniature.

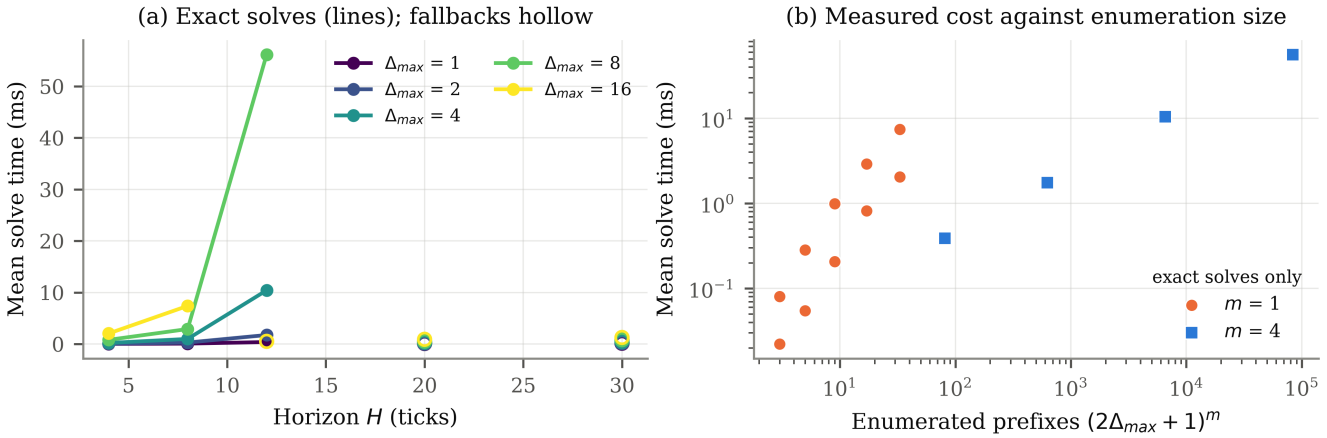


Fig. 12. (a) Solve time against horizon, one line per action-set size; filled markers and lines are exact solves, hollow markers are cells that exceeded the node budget and were solved by the greedy completion, so they are not times for Algorithm 2 and are left unconnected. (b) The exact solves alone against the number of enumerated prefixes, on log axes. Synthetic traces.

I. Exact Versus Greedy-Completion Solving

Section V-H establishes that exact solving is affordable. This section asks the prior question of whether it is necessary, by comparing Algorithm 2 against the cheaper solver it replaced: one that enumerates the admissible first actions but completes each horizon by tracking demand myopically. Both solvers were given the same 1200 plant states, drawn to span the reachable range of replica count, in-flight queue, previous action and forecast slope, and both were scored with the same objective (7)-(9).

Quantity	Value
Dead time τ, horizon H, tracking-relevant actions m	9, 12, 4
Plant states compared	1200
First actions in agreement	1070 (89.17 %)
First actions in disagreement	130 (10.83 %)
Greedy completion attains a worse objective	540 (45.00 %)
Greedy completion attains a better objective	0
Mean objective excess of greedy completion	1.970
Largest objective excess of greedy completion	31.80
Mean relative objective excess	1.148 %
Exact solve time, median / p95 / max	15.17 / 19.66 / 34.25 ms
Greedy solve time, median / p95 / max	0.273 / 0.339 / 0.903 ms

Table XII. Algorithm 2 against enumeration with greedy horizon completion, on the configuration used throughout this paper. Objective excess is in units of the cost (7).

The two solvers choose the same action on 89.17 % of states and differ on the remaining 10.83 %. On 45.00 % of states the greedy completion lands on a plan of strictly higher cost, and on none does it beat Algorithm 2 — the expected signature of an exact

reference, and the reason the comparison is reported in this direction rather than as a symmetric disagreement rate.

Two features of the gap matter more than its size. It is one-sided, so the greedy completion is a relaxation of the same problem rather than a different trade-off; and it is concentrated rather than uniform, with a mean relative excess of 1.148 % against a maximum of 99.5 %. The states where it errs are those whose optimal plan ends the horizon on a large action, which is precisely where zeroing the tail misprices the churn term. A controller built on the greedy completion is therefore not uniformly slightly worse; it is almost always identical and occasionally substantially wrong.

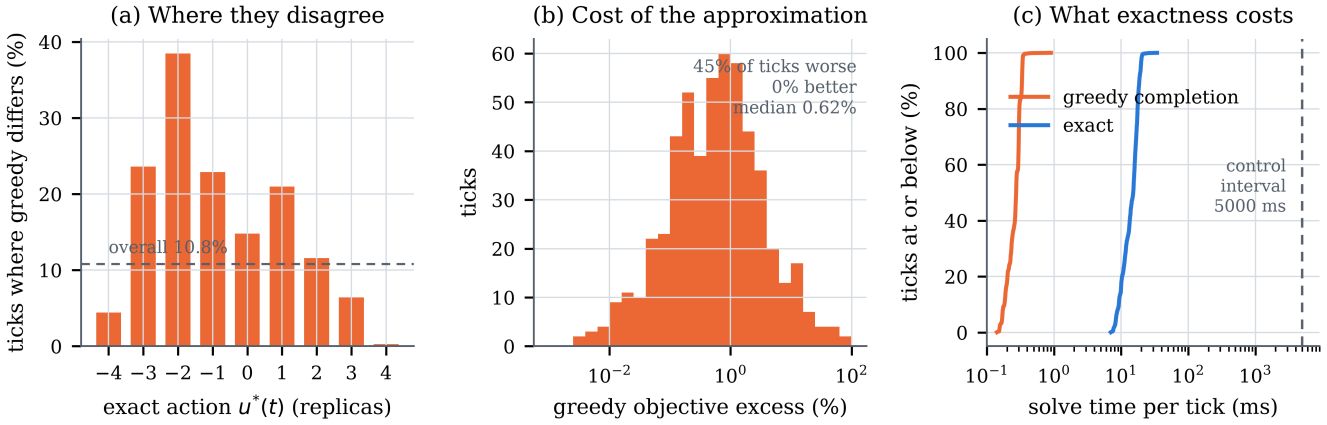


Fig. 13. Algorithm 2 against greedy horizon completion over the same plant states. (a) Disagreement is not uniform over the action space: it peaks on moderate scale-downs and nearly vanishes on large scale-ups. (b) The objective excess is one-sided by construction; no tick favours the greedy completion. (c) Solve time for both, against the control interval.

The disagreement is structured rather than scattered, as Fig. 13(a) shows. It peaks at u*(t) = -2, where the two solvers choose differently on 38.5 % of ticks, and falls to 1.3 % once the exact action is a scale-up of three or more. That shape follows from the mechanism: a large scale-up saturates the rate limit, so every completion agrees on it, whereas a moderate scale-down leaves room for the horizon to be finished several ways and the mispriced tail decides between them. The cases the approximation gets wrong are therefore the ordinary ones, not the extreme ones. Fig. 13(b) shows the excess is one-sided, and Fig. 13(c) the two solve-time distributions against the control interval.

One consequence reaches past the solver and into how the controller is tuned. The control-parameter sweep of Section V-G was run once under each solver, with the same workloads, the same seeds and the same configuration, so the pair isolates what approximating the solve does to a tuning conclusion. Under the greedy completion, raising the churn weight to unity drove the violation rate to 0.009 % and held it there across every larger value tested. Under Algorithm 2 the same sweep sits near 0.42 %, and the largest divergence over the sweep is 0.009 % against 0.43 % at s = 1.5.

The qualitative reading of that sweep is unchanged: raising s still improves service level, capacity and oscillation together, from 36.9 % over-provisioning and 52.6 oscillations at s = 0 to 22.2 % and 33.7 at s = 1. What was not a property of the controller is the size of the effect. The mechanism is the one Section III-E sets out: greedy completion zeroes the tail, a zero tail misprices precisely the term s scales, and raising s magnifies the mispricing. An approximate solver therefore misleads most where the parameter being tuned is the one the approximation distorts, which is not where a reader would think to look for it.

Exactness costs roughly 56 times the compute of the greedy completion. That multiplier is the real price of the claim. In absolute terms the worst observed solve took 34.3 ms, or 0.69 % of the five-second control interval, so the multiplier is large and the absolute cost is not.

That cost is not always payable. The reduction in Section III-E is strong when the dead time is long relative to the horizon, and the memoised recursion is cheap when the dead time is short enough for the pending queue to collapse distinct prefixes onto one state. Between the two lies a band — a queue long enough to keep states distinct and a horizon long enough to make m large — where neither applies, and there the controller falls back to the greedy completion rather than stall the loop. Every result at the configuration used throughout this paper is solved exactly. Three experiments vary the horizon or the dead time far enough to cross the band. In the horizon sweep of Section V-A, 46 of the 60 cells are solved exactly and 14 are not, with each run recording which solver produced it. The fallbacks all sit well above the boundary, where the controller is already at its floor; every cell immediately either side of $H\Delta t = \tau c$ is solved exactly, so the transition itself is measured without the approximation. The same grid on recorded traffic (Section V-Q) has 28 of 120 cells solved by the fallback. In the cold-start sweep of Section V-J the horizon is fixed at H = 30, and the predictive controller was solved exactly on 2 of its 7 cold-start settings (τc = 5 and 15 s); the other 5 are greedy-completion results and are marked so in Table XIII.

J. Robustness to Actuation Delay

Section V-A varied the horizon at fixed cold start. The complementary experiment fixes the horizon at a value that satisfies (6) everywhere and varies the cold start instead, which isolates the controller's tolerance of actuation delay from the question of whether the horizon is adequate. Table XIII reports it and Fig. 14 shows both panels.

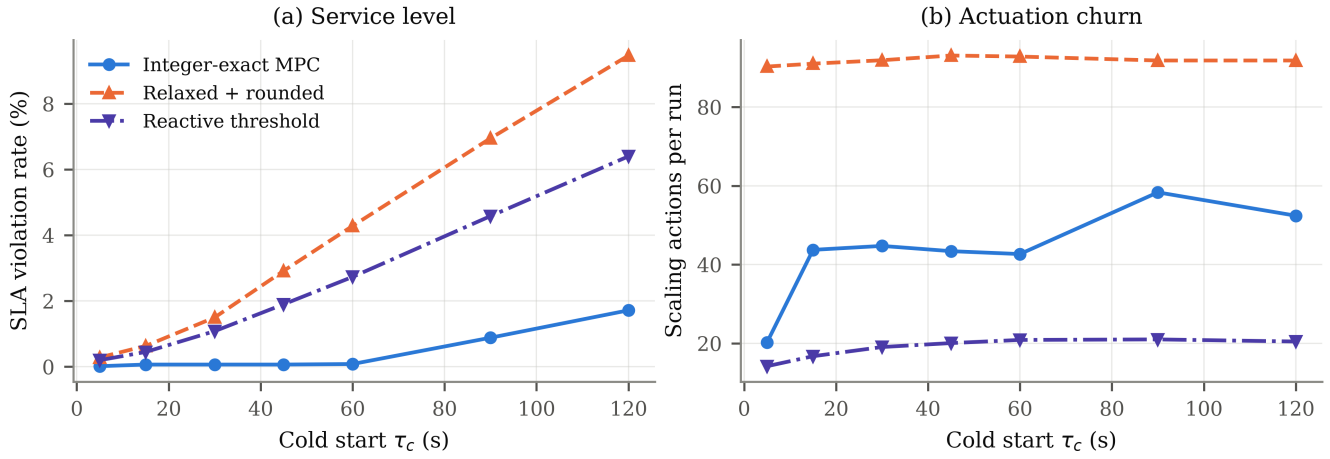
τc (s)	Exact MPC viol. (%)	Rounded viol. (%)	Reactive viol. (%)	Exact actions	Rounded actions	Reactive actions
5	0.02	0.28	0.20	20.3	90.3	14.2
15	0.07	0.63	0.45	43.7	91.0	16.7
30 †	0.07	1.50	1.08	44.7	91.9	19.1
45 †	0.07	2.92	1.90	43.4	93.1	20.1
60 †	0.08	4.28	2.73	42.7	92.8	20.9
90 †	0.88	6.95	4.58	58.3	91.8	21.0
120 †	1.72	9.47	6.40	52.4	91.8	20.5

Table XIII. Sensitivity to cold-start latency with the horizon fixed at H = 30 (150 s lookahead), which satisfies (6) at every value shown. Synthetic traces, 3 families × 5 seeds = 15 runs per cell. † At this dead time an H = 30 plan exceeds the exact solver's budget, so the predictive controller was solved by the greedy completion on every tick; those entries are not integer-exact.

Only the τc = 5 s and 15 s rows of this sweep are solved exactly. From τc = 30 s upward, an H = 30 plan falls in the band of Section V-I where the exact solve is unaffordable, and the predictive controller ran on the greedy completion. What follows is therefore a statement about the predictive controller with an adequate horizon, and only its two shortest delays are integer-exact. With an adequate horizon the predictive controller is close to insensitive to actuation delay over the practical range. Violations stay between 0.02 % and 0.08 % as the cold start grows from 5 s to 60 s, a factor of twelve in delay. The reactive controller degrades steadily over the same range, from 0.20 % to 2.73 %, and the rounded controller degrades faster still, from 0.28 % to 4.28 %. At the longest delay tested, 120 s, the three sit at 1.72 %, 6.40 % and 9.47 % respectively.

This experiment measures what the ablation of Section V-E could not. In that ablation, disabling the dead-time term left service level statistically unchanged, because the horizon remained long enough that the optimiser could compensate implicitly. Here the dead time itself is the variable, and the separation between the reactive and the exact controller grows monotonically with it, from 0.18 to 4.68 points. Dead-time tolerance is therefore a property of the predictive formulation as a whole rather than of the explicit delay term in (5), and the paper's claim is stated at that level.

The churn panel adds a second observation. The rounded controller's action count is essentially flat at roughly 93 per run regardless of delay, because its oscillation is driven by fractional rounding rather than by the plant. The exact controller's action count varies with delay, as a controller responding to the plant should.



Horizon fixed at $H = 30$ (150 s lookahead), which satisfies Eq. (6) at every τ_c shown. Synthetic traces, 3 families × 5 seeds.

Fig. 14. Service level and churn against cold-start latency at a fixed adequate horizon. Synthetic traces. The predictive controller's points at τc ≥ 30 s were solved by the greedy completion (Table XIII).

K. Per-Workload Behaviour

The aggregate figures of Section V-D average over 7 demand shapes that stress the controller differently. Table XIV disaggregates them, and the variation is large enough that the aggregate alone would be misleading.

Workload	Method	Violation (%)	Over-prov. (%)	Actions	Oscillations	p95 (ms)
Diurnal	Exact MPC	0.00 ± 0.00	18.5 ± 1.7	37.8	32.5	93
	Rounded	0.30 ± 0.35	3.8 ± 0.3	119.0	109.7	110
	Reactive	0.00 ± 0.00	33.7 ± 16.9	12.4	2.5	90
	Static	0.00 ± 0.00	47.9 ± 0.6	0.0	0.0	83
Burst	Exact MPC	0.00 ± 0.00	22.6 ± 2.4	49.8	35.2	94
	Rounded	5.17 ± 1.18	4.2 ± 0.7	84.3	52.1	168
	Reactive	2.45 ± 0.51	87.1 ± 1.3	27.2	3.0	158
	Static	0.00 ± 0.00	158.4 ± 10.9	1.0	0.0	78
Ramp	Exact MPC	0.00 ± 0.00	13.6 ± 0.8	35.9	25.6	91
	Rounded	0.25 ± 0.29	2.8 ± 0.3	116.0	101.1	107
	Reactive	0.03 ± 0.08	41.3 ± 2.5	15.5	1.0	88
	Static	0.00 ± 0.00	97.2 ± 1.3	1.0	0.0	80
Step	Exact MPC	0.28 ± 0.14	9.9 ± 0.5	32.2	26.9	101
	Rounded	2.38 ± 0.21	3.6 ± 0.3	64.6	57.4	217
	Reactive	2.50 ± 0.00	13.6 ± 1.3	6.7	1.0	213
	Static	0.00 ± 0.00	89.6 ± 8.9	1.0	0.0	81
Sawtooth	Exact MPC	4.00 ± 1.70	52.2 ± 11.5	52.0	30.0	97
	Rounded	0.35 ± 0.58	2.8 ± 0.1	107.5	82.4	105
	Reactive	0.00 ± 0.00	72.6 ± 11.8	18.6	5.4	84
	Static	0.00 ± 0.00	61.8 ± 6.9	1.0	0.0	81
Worldcup98	Exact MPC	0.00 ± 0.00	36.5 ± 13.9	51.3	50.0	106
	Rounded	0.05 ± 0.16	3.1 ± 2.6	57.9	57.1	123
	Reactive	0.00 ± 0.00	14.8 ± 16.4	1.2	0.2	104
	Static	0.00 ± 0.00	19.1 ± 16.4	0.0	0.0	99
Worldcup98_testpeak	Exact MPC	0.00 ± 0.00	11.9 ± 2.0	51.7	47.8	87
	Rounded	0.00 ± 0.00	2.3 ± 0.3	156.6	127.4	93
	Reactive	0.00 ± 0.00	11.5 ± 6.5	3.0	0.6	86
	Static	0.00 ± 0.00	18.3 ± 3.7	0.0	0.0	83

Table XIV. Per-workload breakdown. Five synthetic families and two recorded WorldCup98 scalings, 10 seeds or windows per row, oracle predictor; values are mean ± one standard deviation.

The burst family separates the methods most sharply. There the rounded controller reaches 5.17 % violations against 0.00 % for the exact controller, the largest gap in the table, because a burst drives the pool through many integer boundaries in quick succession and every crossing is an opportunity for the rounding error to change the decision. The diurnal and ramp families, whose demand changes slowly relative to the control interval, show almost no separation at all — 0.30 % against 0.00 % on diurnal traffic. The advantage of exact solution is therefore concentrated in workloads with fast transitions, which is also where an auto-scaler is most needed and least likely to perform well.

The sawtooth family is the one case where the exact controller violates more than the rounded one; on sawtooth traffic it reaches 4.00 % against 0.35 %, with a standard deviation across seeds of 1.70. A sawtooth resets demand discontinuously every ninety ticks. One plausible mechanism is that the churn penalty resists the repeated scale-down the shape demands, so the controller spends part of each period recovering; no experiment here isolates it. The failure is reported rather than averaged away.

The two recorded families are easier than the synthetic ones at the default operating point. On WC98/trace the exact, rounded and reactive controllers violate on 0.00 %, 0.05 % and 0.00 % of ticks, carrying 36.5 %, 3.1 % and 14.8 % excess capacity; On WC98/test the exact, rounded and reactive controllers violate on 0.00 %, 0.00 % and 0.00 % of ticks, carrying 11.9 %, 2.3 % and 11.5 % excess capacity. On recorded traffic the reactive baseline is not the over-provisioned method it is on the synthetic families: on WC98/trace and WC98/test it holds less excess capacity than the exact controller at no measurable service-level cost.

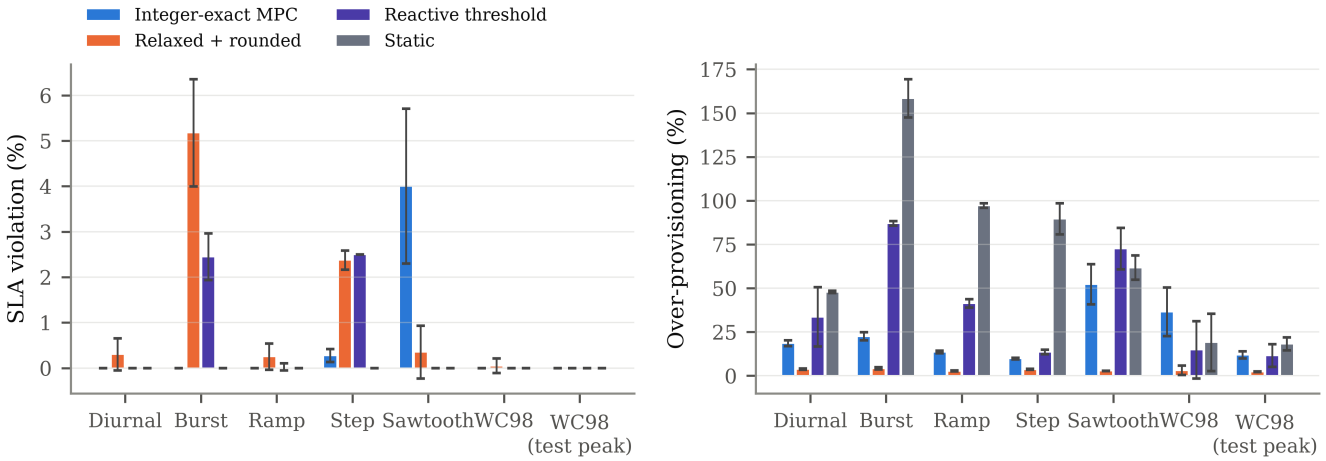


Fig. 15. Per-workload service level and capacity cost. Whiskers are one standard deviation over 10 seeds or windows. Five synthetic families and two recorded WorldCup98 scalings.

L. Decision-Level Analysis

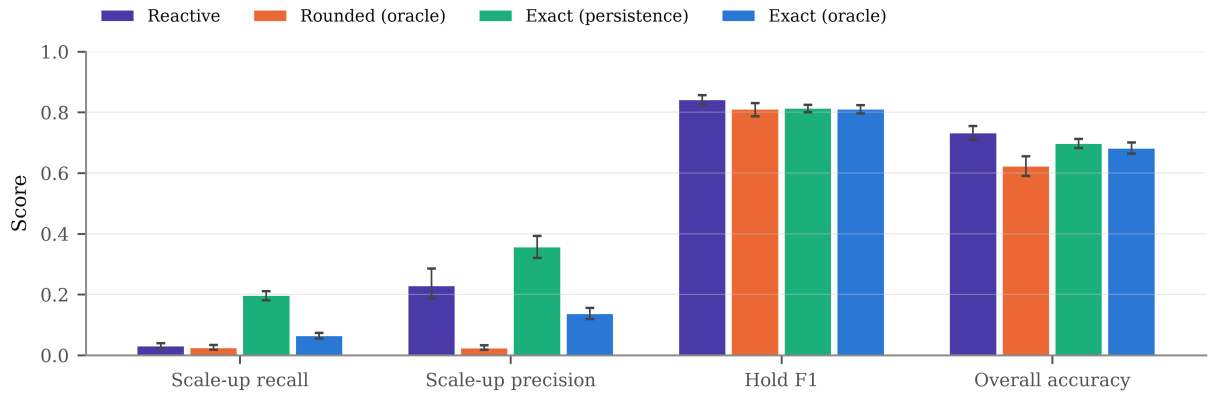
Treating each tick as a three-way classification into scale-up, hold and scale-down against the requirement implied by the observed load gives a finer view than the aggregate rates. Table XV reports it with bootstrap intervals and Fig. 16 shows it. The numbers are low in absolute terms for every method, and the reason is worth stating because it bears on how such metrics should be read.

Method	Scale-up recall	Scale-up precision	Hold F1	Accuracy
Exact MPC (oracle)	0.065 [0.055, 0.073]	0.137 [0.119, 0.156]	0.811	0.683
Exact MPC (persistence)	0.197 [0.181, 0.211]	0.357 [0.320, 0.393]	0.814	0.698
Relaxed + rounded	0.025 [0.018, 0.034]	0.025 [0.018, 0.033]	0.811	0.623
Reactive threshold	0.031 [0.024, 0.040]	0.230 [0.187, 0.286]	0.842	0.733

Table XV. Per-tick decision classification against the observed requirement. Five synthetic families and two recorded WorldCup98 scalings, 70 runs per row; intervals are 95 % bootstrap.

Scale-up recall is low for every method, between 0.025 and 0.197. This is a property of the ground-truth definition rather than of the controllers. The requirement series is computed per tick from the instantaneous arrival rate and therefore changes on most ticks, including by a single replica in response to noise, whereas a well-behaved controller deliberately does not track every such change. A controller that matched this ground truth tick for tick would be the oscillating controller the design is intended to avoid. The metric is reported for completeness and because it is standard in the literature, but it should not be read as a quality ranking, and the paper does not use it as one.

The one comparison it does support is between predictors at fixed control law. Moving from persistence to perfect foresight changes scale-up recall from 0.197 to 0.065 and precision from 0.357 to 0.137. The persistence predictor scores higher on both, which at first appears paradoxical. It follows from the same definitional issue: persistence reacts to the most recent observation and therefore produces decisions that correlate with a per-tick ground truth derived from that same observation, while the oracle acts on demand that has not yet arrived and is scored as wrong until it does. The aggregate service-level figures in Table VII, where the oracle is ahead, are the more meaningful comparison.



Each tick classified into scale-up / hold / scale-down against the observed requirement. Synthetic and recorded traces, 70 runs per bar, 95 % bootstrap intervals.

Fig. 16. Per-tick decision classification with 95 % bootstrap intervals. Five synthetic families and two recorded WorldCup98 scalings.

M. Response to Predictor Quality

The bracketing protocol of Section IV-D bounds the predictor's contribution but leaves the interior unmeasured, and a real forecaster lives in that interior. To fill it, a blended predictor was constructed that returns a convex combination of the two brackets, with weight w on perfect foresight and 1 − w on persistence. Sweeping w traces the controller's response to predictor quality without requiring any particular model to be trained. Table XVI reports the sweep and Fig. 17 shows it.

w	Exact MPC violation (%)	Exact MPC over-prov. (%)	Exact MPC actions	Rounded violation (%)	Rounded over-prov. (%)	Rounded actions
0.0	0.76	26.9	53.7	1.26	3.7	91.1
0.2	0.73	24.5	49.9	1.05	3.6	65.0
0.4	0.47	22.6	46.9	0.86	3.5	49.2
0.6	0.23	19.7	41.3	0.88	3.3	48.5
0.8	0.04	18.2	40.1	1.00	3.1	64.2
0.9	0.24	19.2	40.4	1.14	3.0	76.5
1.0	0.69	22.4	43.1	1.33	2.9	88.8

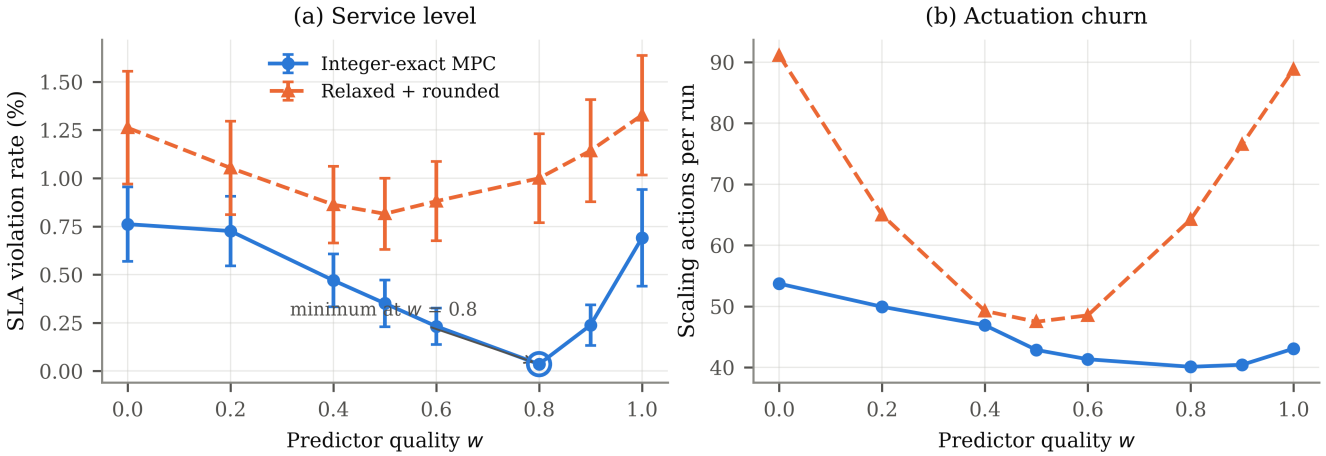
Table XVI. Response to predictor quality. w = 0 is the persistence bracket and w = 1 the perfect-foresight bracket. Five synthetic families and two recorded WorldCup98 scalings, 42 runs per cell, pooled.

The response is not monotone, and that is the substantive finding. For the integer-exact controller the violation rate falls from 0.76 % at pure persistence to a minimum of 0.04 % at w = 0.8, then rises again to 0.69 % at perfect foresight. The rounded controller shows the same shape with its minimum at w = 0.5. A perfectly accurate forecast is therefore not the best forecast for either controller on these workloads.

The mechanism is visible in the churn panel of Fig. 17. A perfect forecast reproduces every fluctuation in demand, including the noise, and the controller tracks it; actions rise from 40.1 at the optimum to 43.1 at w = 1. Blending in a persistence component low-pass filters the forecast, and the resulting slight lag behaves like a small damping term. The controller acts less often, holds a steadier pool, and ends up violating less. The effect is stronger for the rounded controller, whose action count rises 1.87-fold from 47.5 at its optimum to 88.8 at perfect foresight, because its fractional oscillation compounds with the unfiltered signal.

Two consequences follow for how such systems should be built and evaluated. First, optimising a forecaster for accuracy alone optimises the wrong objective, which is the methodological point the paper makes in Section I and here measures directly: an accuracy improvement that sharpens the predicted signal can degrade the closed loop. Second, the bracketing protocol is conservative in an unexpected direction. Because the interior of the range outperforms both endpoints, the reported bracket does not contain the best achievable result, and a trained forecaster of intermediate sharpness could perform better than either bracket rather than between them. The trained forecaster of Section V-Q is better than both brackets on 13 of its 24 controller–metric comparisons and worse than both on 4; whether its advantage comes from the damping mechanism described here was not tested.

The result should not be over-read. The table pools five synthetic families, whose noise is additive and Gaussian, with two scalings of one recorded trace, and does not separate them; the damping benefit depends on the high-frequency content being uninformative. On a workload whose high-frequency content carries real demand, filtering it away would cost service level rather than save it. What the experiment establishes is that the relationship between forecast accuracy and control performance is not monotone, not the location of the optimum in general.



$w = 0$ is persistence, $w = 1$ is perfect foresight. Synthetic and recorded traces, 7 families × 6 seeds per point; bars are the standard error.

Fig. 17. Controller response to predictor quality. The violation-rate minimum for both controllers lies strictly inside the bracket rather than at perfect foresight. Five synthetic families and two recorded WorldCup98 scalings, pooled.

N. Configuration Guidance

The measurements support a short configuration procedure, stated here because the parameters interact and the order matters.

- Measure the cold-start latency of the deployed image, including image pull on a cold cache, readiness probing and load-balancer registration. This single measurement determines the horizon through (6) and is the one parameter that cannot be guessed.
- Set the horizon to the smallest value satisfying $H\Delta t \geq \tau c$, then add one interval of slack. Section V-A shows the penalty for an under-sized horizon is severe and the benefit of oversizing it is negligible, so the asymmetry favours erring long.
- Set the cooldown next. Of the parameters swept in Section V-G it has the largest effect on churn: removing it raises actions from 44.5 to 272.1 per run. Lengthening it to 120 s lowered both actions (26.1) and violations (0.15 %) in the pooled sweep, at 2.3 points more over-provisioning; the sweep was pooled across families and was not repeated per workload.
- Set the churn weight to unity or above. Raising s from 0 to 1 improved violations, over-provisioning and oscillations together, and s = 2, the largest value swept, cut actions further to 25.8 at 0.38 % violations. The upper end of the useful range was not reached by the sweep.
- Leave the deadband at zero or one, which behave identically. $\theta = 2$ cuts actions from 44.5 to 7.6 but costs 0.15 points of violation and 4.6 of over-provisioning. Do not set the cost asymmetry to unity: $q^+/q^- = 1$ gives 0.85 % violations against 0.39 % at 20.

The procedure assumes a single replica pool with the plant characteristics described in Sections IV-A and IV-B and should be re-validated before transfer to a different deployment, particularly one in which several services share hosts.

O. Decision Trace

Aggregate metrics describe behaviour but not mechanism. Table XVII reproduces sixteen consecutive control ticks from a single run, taken from the rise of the first burst on the burst trace, with every intermediate quantity the controller computes. It is included because the differences the aggregate tables report are visible here as individual decisions, and because an auto-scaler that cannot be read tick by tick cannot be debugged.

Tick	λ (req/s)	Observed u	Forecast u at h=12	Demand Ñ(t+1)	MPC u*	Rounded u	After gates	Applied u	N(t)	Required (ground truth)	Excess
106	121	0.605	0.825	5	0	-1	0	0	6	5	+1
107	113	0.565	0.922	5	0	-1	0	0	6	5	+1
108	124	0.619	0.884	6	0	0	0	0	6	5	+1
109	138	0.690	0.970	6	1	0	1	1	6	6	0
110	133	0.555	0.878	6	0	-1	0	0	7	6	+1
111	152	0.631	0.829	6	0	-1	0	0	7	6	+1
112	145	0.603	0.902	6	0	-1	0	0	7	6	+1
113	141	0.586	0.947	6	1	-1	1	1	7	6	+1
114	145	0.605	0.941	7	-1	-1	0	0	8	6	+2
115	157	0.653	0.911	6	-1	-2	0	0	8	7	+1
116	153	0.638	0.944	7	-1	-1	0	0	8	6	+2
117	157	0.654	0.991	7	-1	-1	0	0	8	7	+1
118	165	0.687	0.948	8	0	0	0	0	8	7	+1
119	184	0.659	0.800	7	0	-1	0	0	8	8	0
120	177	0.631	0.907	8	1	0	1	1	8	7	+1
121	194	0.693	0.843	9	-1	0	0	0	9	8	+1

Table XVII. Sixteen consecutive control ticks from the burst trace, seed 1001, oracle predictor. The ground-truth column is the replica count the instantaneous arrival rate required at the target utilisation; it is computed after the fact and is not available to the controller. The excess column is N(t) minus that requirement.

The two controllers disagree on 11 of the 16 ticks shown, which is consistent with the rates in Table V. The pattern of disagreement is informative: the rounded controller proposes a scale-down on most ticks where the exact controller proposes holding. Its relaxed solution sits slightly below the current count because the instantaneous demand is slightly below it, and rounding converts that fractional preference into a whole replica. The exact controller, evaluating the same horizon under the integrality constraint, finds that removing a replica costs more over the horizon than it saves on the current tick.

The gate column shows the cooldown active on 5 ticks, each time converting a proposed scale-down to a hold. Ticks 113 through 117 are the clearest case: the optimiser proposes removing a replica on five consecutive ticks and the cooldown suppresses all five, because a scale-up occurred recently. The arrival rate then rises through tick 121, which retrospectively makes every one of those suppressed scale-downs the wrong action. This is the mechanism behind the sensitivity result of Section V-G, where the cooldown dominated stability, seen at the level of the individual decisions it changes.

The excess column averages 1.00 replicas over the window, which is the over-provisioning the aggregate tables report expressed per tick. It is positive on almost every tick rather than alternating in sign, which identifies it as a systematic bias of the controller rather than tracking noise. The asymmetric tracking weight of (8) and the cooldown both push in that direction, and Section V-G shows the cooldown is the larger contributor.

One caveat applies to reading this table. The ground-truth column is derived from the instantaneous arrival rate, so it changes on nearly every tick, including by a single replica in response to noise. A controller that matched it exactly would be the oscillating controller the design is intended to avoid, and the excess column should be read as a measure of conservatism rather than of error. This is the same definitional caveat that applies to the decision classification of Section V-L.

P. Positioning Against Prior Work

Table XVIII places the present work against the reviewed systems on the three axes the experiments address. The comparison is of mechanism, not of reported numbers: none of the systems in the table was reimplemented, their published results were obtained on different plants and workloads, and no numerical claim is made against them.

Property	Threshold [1]	Forecast-driven [2,3,8]	Optimisation [4,5,6]	Hybrid [7,11]	This work
Decision variable is integer	yes	varies	no, rounded	varies	yes, solved exactly
Actuation delay modelled	no	no	no	partially	yes, as plant dead time
Horizon tied to delay	n/a	no	no	no	yes, Eq. (6), measured
Decision churn penalised	via stabilisation window	no	no	no	yes, in the objective
Churn reported as a metric	no	no	no	no	yes
Hard pool and rate constraints	yes	varies	yes	varies	yes
Conservatism mechanism	static threshold	none	chance constraint	feedback term	clipped-integral margin
Evaluated on real hardware	yes	varies	yes	yes	no, simulated plant

Table XVIII. Mechanism comparison. Entries describe design choices reported by the cited work, not relative performance. The final row records the principal weakness of the present evaluation.

Three columns separate this work from the rest — exact integrality, a horizon tied to measured delay, and churn treated as both a penalty and a reported metric — and those are precisely the three the experiments quantify. The last row runs the other way: every other system in the comparison was evaluated on real infrastructure, and this one was not.

Q. Recorded Traffic and a Trained Forecaster

Every result up to this point was obtained on generated traces with bracketing predictors. This subsection repeats the central measurements on recorded request traffic and replaces the brackets with a forecaster trained on that traffic. It reports where the trained forecaster fell relative to the brackets, and which of the earlier findings survive the change of workload. A finding is

said to survive only when the rule stated with Table XXI is met; findings that fail it are reported as failing.

The trace

The recorded workload is the 1998 FIFA World Cup web-site access log [11], days 41–50 (1998-06-05 to 1998-06-14, Paris time), 41 log files and 250,257,178 requests. The ten days are contiguous, so the chronological splits cut one unbroken series, and they span the start of the tournament: the first five are pre-tournament days and the opening match was played on 10 June, so the subset contains a diurnal cycle, a rise in base load and match-time bursts. The subset was fixed before any controller was run on it. Every file was decoded and checked against the reference counts shipped with the archive. Requests were binned into the 5 s control interval and divided by 5 to give an arrival rate; no smoothing or outlier removal was applied. The raw peak is 2316.2 req/s (10 June 19:28), well beyond the 640 req/s that the simulated pool can serve at its upper bound.

That mismatch is handled by rescaling, which preserves shape but not absolute level. Two scalings are reported, because neither is satisfactory alone. The first, WC98/trace, multiplies the whole series by 0.2349 so that the trace peak is 85 % of pool capacity; because the peak falls in the training split, the test split then peaks at only 135.0 req/s and the pool is lightly loaded throughout. The second, WC98/test, multiplies the first by a further 4.031 so that the test-split peak is 85 % of capacity. The forecaster never sees this factor: its inputs and targets are standardised with training-split statistics, under which a constant multiple cancels, so the same frozen model serves both families and its output is multiplied back. Choosing a scale from the test peak does use one statistic of the test split, and this is stated as a limitation in Section VII.

The series is split chronologically 60/20/20: training on 5–10 June, validation on 11–12 June and test on 13–14 June. The test split is a Saturday and Sunday, so it is not a random sample of the week. Every closed-loop run on recorded traffic uses a 400-tick window drawn from the test split, and the forecaster is checked at every tick to have been given only past observations. Fig. 18 shows the series and the splits. The trace also contains a brief dip at each local midnight, where the logging servers appear to have rotated their files; it is left in the data rather than repaired, and one test window contains it.

Cutting windows from the middle of a recorded day exposed a defect in the harness that the synthetic traces had hidden. Every run used to open with four replicas. The synthetic generators begin near their base level, which four replicas can serve, but a recorded window begins at whatever load the day has reached. In the first recorded runs, with that fixed start, the reactive controller on WC98/test incurred 92 violation ticks over 9 windows, 92 of them in the first 30 ticks, and the exact controller 92, 92 of them early: the measured violations were the start-up transient, identical for every controller, and said nothing about control. The start state was therefore changed, for every workload alike, to four replicas or the requirement (2) of the first tick, whichever is larger. Under that rule the counts are 0 (0 early) and 0 (0 early). Across the synthetic families the rule changes the start of 5 of 45 measured windows, which open slightly above what four replicas cover at the target. The change was made after this diagnosis and before any trained-arm result had been produced, and every experiment in this paper, synthetic and recorded, was re-run under it.

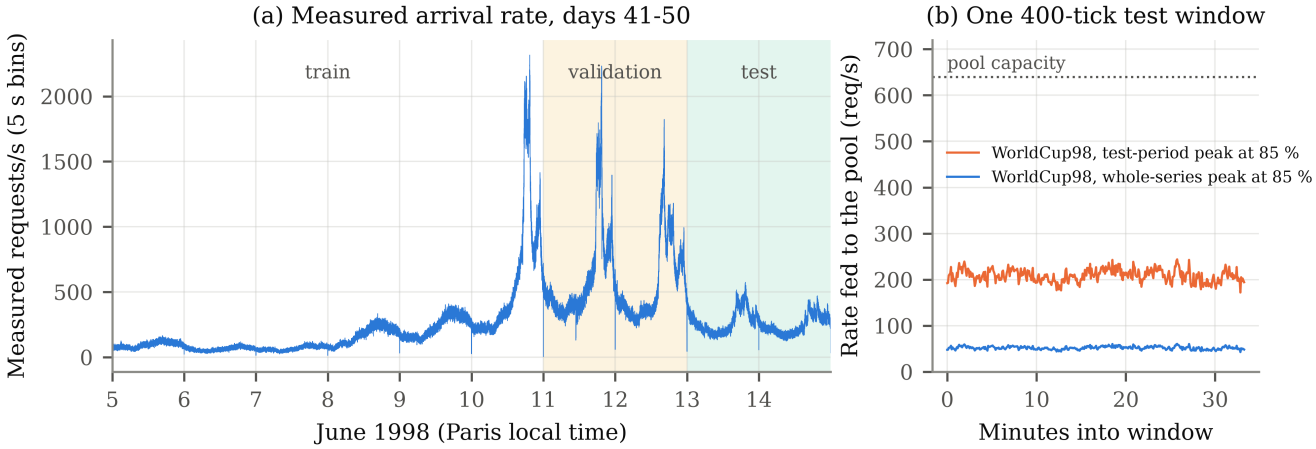


Fig. 18. (a) The measured WorldCup98 arrival rate, days 41–50, at Δt = 5 s before scaling, with the chronological training, validation and test splits shaded. (b) One test window as fed to the pool under each scaling, against pool capacity. Recorded traffic [11].

Forecast accuracy

The proposed forecaster is a dilated temporal convolutional network with 188,504 parameters, trained on the arrival rate with a 48-tick history, a 12-step horizon and the asymmetric loss of the forecasting module (κ = 2.5), for 17 epochs before early stopping on the validation split. Its prediction of arrivals is converted to the utilisation and memory series the controller consumes through the same capacity law used for the oracle. Four neural baselines with parameter counts within 0.3 % of the TCN, an ARIMA model selected by AIC, and persistence were evaluated on the identical 34,501 test origins. Table XIX reports the result and Fig. 19 the full per-horizon curves.

Model	Parameters	Epochs	nMAE h=1 (%)	nMAE h=6 (%)	nMAE h=12 (%)	MAE h=12 (req/s)	Bias h=12 (req/s)	Under-forecast h=12 (%)
Persistence	0	—	4.50	6.19	6.72	4.07	0.01	49.7
ARIMA	5	—	4.03	5.17	5.62	3.40	0.02	49.1
TCN (proposed)	188504	17	4.64	6.09	6.70	4.06	2.64	26.1
LSTM	189128	9	4.46	5.87	6.41	3.89	-0.35	48.8
GRU	188148	9	4.45	5.50	5.81	3.52	0.71	40.4
Bi-LSTM	188280	9	4.64	5.72	6.08	3.69	0.79	40.6
Transformer	188132	18	4.72	5.59	5.75	3.49	1.02	38.9

Table XIX. Forecast error on the WorldCup98 test split, 34,501 origins, one training seed per model. nMAE is mean absolute error as a percentage of the mean test arrival rate; bias is mean forecast minus actual; under-forecast is the share of forecasts below the actual value. Rates are in WC98/trace units.

The proposed model is not the most accurate. At the full horizon it ranks 6 of 7, with an nMAE of 6.70 % against 5.62 % for ARIMA, the best model, and 6.72 % for persistence; it is more accurate than persistence at 10 of the 12 horizons. At one step it is less accurate than persistence (4.64 % against 4.50 %). The best neural model is Transformer at 5.75 %, and ARIMA, with 5 parameters, is more accurate than the neural model in 60 of the 60 model–horizon pairs. This is consistent with the observation of Fog et al. [3] that classical models are competitive on smooth or strongly seasonal traffic.

The neural models share one further property: they are biased upward. The TCN over-forecasts by 2.64 req/s on average at the full horizon and falls below the actual rate on only 26.1 % of forecasts, against 49.7 % for persistence and 49.1 % for ARIMA. That is the intended effect of the asymmetric loss, which charges under-prediction 2.5 times as heavily, and every neural model was trained with it, so the comparison among architectures is like for like. The comparison against ARIMA and persistence is not: those are fitted to be unbiased, and part of their accuracy advantage is the error the asymmetric loss accepts deliberately. The neural models were trained once each; no seed variance is reported, and the ranking among them should not be read as stable.

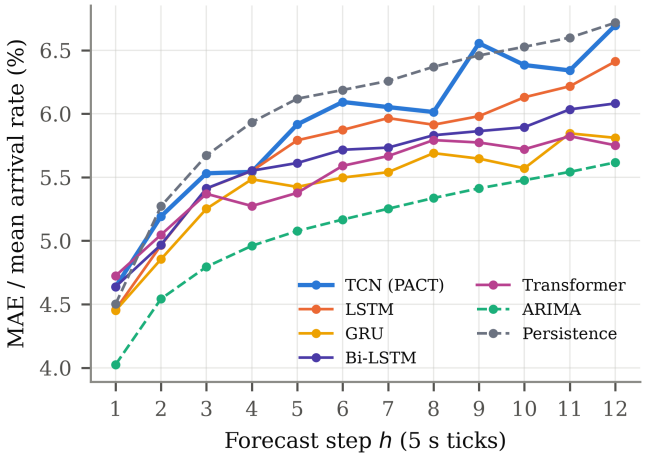


Fig. 19. Forecast error (nMAE) against forecast step for every model on the WorldCup98 test split. One training seed per neural model.

The trained forecaster against the brackets

The frozen TCN was then placed in the loop. Both controllers were run with each of the three predictors on every disjoint 400-tick window of the test split, 86 windows per family, and the reactive baseline was run on the same windows. Table XX compares the trained arm with the two brackets for each metric, with paired Wilcoxon tests Holm-corrected across the whole table.

Family	Controller	Metric	Oracle	Trained	Persistence	Position	p vs. oracle (Holm)	p vs. persist. (Holm)
WC98/trace	Exact	Violation (%)	0.02	0.00	0.01	better than both	0.625	1.000
WC98/trace	Exact	Over-prov. (%)	32.62	37.35	39.18	between	< 0.001	0.374
WC98/trace	Exact	Actions	50.95	50.43	57.34	better than both	0.625	< 0.001
WC98/trace	Exact	Oscillations	49.74	48.80	55.26	better than both	0.388	< 0.001
WC98/trace	Exact	Cost /1k req	60.67	62.92	63.85	between	< 0.001	0.178
WC98/trace	Exact	Mean p95 (ms)	106.27	102.40	103.24	better than both	< 0.001	0.015
WC98/trace	Rounded	Violation (%)	0.20	0.05	0.17	better than both	0.004	0.012
WC98/trace	Rounded	Over-prov. (%)	2.34	3.80	1.83	worse than both	< 0.001	< 0.001
WC98/trace	Rounded	Actions	42.74	22.15	37.30	better than both	< 0.001	< 0.001
WC98/trace	Rounded	Oscillations	41.65	21.34	36.06	better than both	< 0.001	< 0.001
WC98/trace	Rounded	Cost /1k req	46.32	47.65	47.00	worse than both	< 0.001	< 0.001
WC98/trace	Rounded	Mean p95 (ms)	119.93	112.58	117.72	better than both	< 0.001	< 0.001
WC98/test	Exact	Violation (%)	0.00	0.00	0.00	between	1 (identical)	1 (identical)
WC98/test	Exact	Over-prov. (%)	11.23	13.30	16.06	between	< 0.001	< 0.001
WC98/test	Exact	Actions	48.78	45.58	55.99	better than both	0.023	< 0.001
WC98/test	Exact	Oscillations	43.79	41.76	47.62	better than both	0.543	< 0.001
WC98/test	Exact	Cost /1k req	44.90	45.76	46.95	between	< 0.001	< 0.001
WC98/test	Exact	Mean p95 (ms)	86.49	85.65	85.50	between	< 0.001	0.178
WC98/test	Rounded	Violation (%)	0.03	0.03	0.03	between	1 (identical)	1.000
WC98/test	Rounded	Over-prov. (%)	2.32	3.80	2.45	worse than both	< 0.001	< 0.001
WC98/test	Rounded	Actions	151.63	72.50	151.87	better than both	< 0.001	< 0.001
WC98/test	Rounded	Oscillations	122.81	62.77	120.23	better than both	< 0.001	< 0.001
WC98/test	Rounded	Cost /1k req	40.62	41.94	41.58	worse than both	< 0.001	< 0.001
WC98/test	Rounded	Mean p95 (ms)	94.06	89.76	93.00	better than both	< 0.001	< 0.001

Table XX. The trained TCN against both brackets on recorded traffic, 86 test windows per family. Lower is better for every metric. "Between" means the trained mean lies inside the interval spanned by the two bracket means. "1 (identical)" marks paired runs that were identical, which are excluded from the Holm correction.

The trained arm landed between the brackets on 7 of the 24 comparisons, better than both on 13 and worse than both on 4. One of the "between" comparisons is degenerate, with all three arms at the same value (WC98/test, exact, violation), so the trained arm sat inside a non-degenerate bracket on 6 of 23. The comparisons outside the bracket are: WC98/trace, exact, violation (better than both; trained 0.00, oracle 0.02, persistence 0.01); WC98/trace, exact, actions (better than both; trained 50.43, oracle 50.95, persistence 57.34); WC98/trace, exact, oscillations (better than both; trained 48.80, oracle 49.74, persistence 55.26); WC98/trace, exact, mean p95 (better than both; trained 102.40, oracle 106.27, persistence 103.24); WC98/trace, rounded, violation (better than both; trained 0.05, oracle 0.20, persistence 0.17); WC98/trace, rounded, over-prov. (worse than both; trained 3.80, oracle 2.34, persistence 1.83); WC98/trace, rounded, actions (better than both; trained 22.15, oracle 42.74, persistence 37.30); WC98/trace, rounded, oscillations (better than both; trained 21.34, oracle 41.65, persistence 36.06); WC98/trace, rounded, cost /1k req (worse than both; trained 47.65, oracle 46.32, persistence 47.00); WC98/trace, rounded, mean p95 (better than both; trained 112.58, oracle 119.93, persistence 117.72); WC98/test, exact, actions (better than both; trained 45.58, oracle 48.78, persistence 55.99); WC98/test, exact, oscillations (better than both; trained 41.76, oracle 43.79, persistence 47.62); WC98/test, rounded, over-prov. (worse than both; trained 3.80, oracle 2.32, persistence 2.45); WC98/test, rounded, actions (better than both; trained 72.50, oracle 151.63, persistence 151.87); WC98/test, rounded, oscillations (better than both; trained 62.77, oracle 122.81, persistence 120.23); WC98/test, rounded, cost /1k req (worse than both; trained 41.94, oracle 40.62, persistence 41.58); WC98/test, rounded, mean p95 (better than both; trained 89.76, oracle 94.06, persistence 93.00). Against persistence, 18 of the differences are significant after correction, 14 in the trained arm’s favour and 4 against it; against the oracle, 18 are.

A result outside the bracket is not in itself evidence of an error, because Section V-M already showed that a predictor between the two extremes can beat both. It was nonetheless checked rather than assumed. Each forecast is computed from the history window ending at the current tick, the harness asserts at every tick that the observation it receives is the recorded value for that tick, every window starts after the validation split ends, and the checkpoint records the checksum of the trace it was trained on, which the harness verifies before use. None of these checks failed in any run. The outcome was reported as measured and the model was not re-tuned after it was seen.

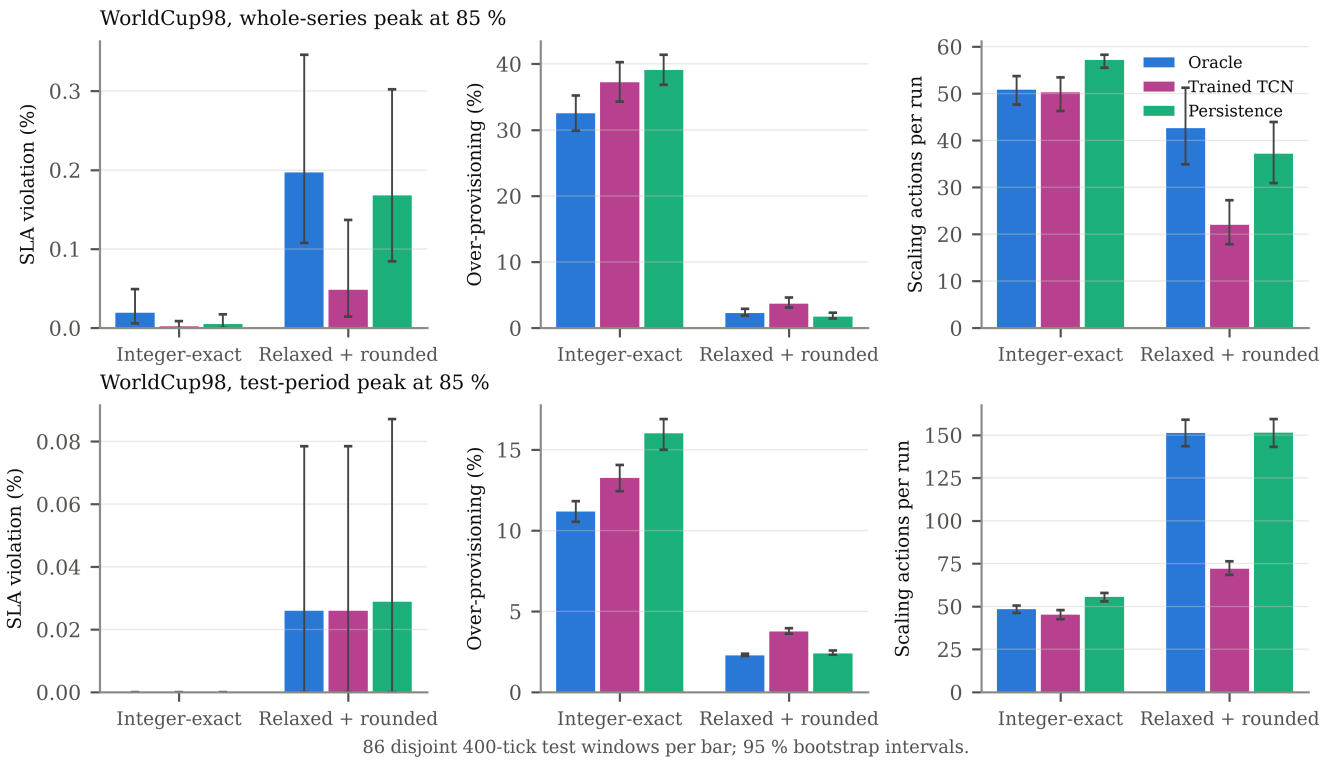


Fig. 20. The trained arm against both brackets on recorded traffic: violation, over-provisioning and scaling actions, with 95 % bootstrap intervals over test windows. Table XX gives every metric.

Which findings survive

The horizon–dead-time sweep of Section V-A was repeated on each recorded family with the oracle predictor, 9 test windows per cell (Fig. 21). On WC98/trace the reactive baseline never violated in any cell, so the plant was not stressed enough to test the transition. The predictive controller violated in 14 of the 60 cells, all above the boundary, on at most 4 of the 3600 ticks in a cell; read literally that is the opposite of the synthetic result, but it rests on a handful of ticks. On WC98/test no cell produced a single violation for either controller, so the transition cannot be observed.

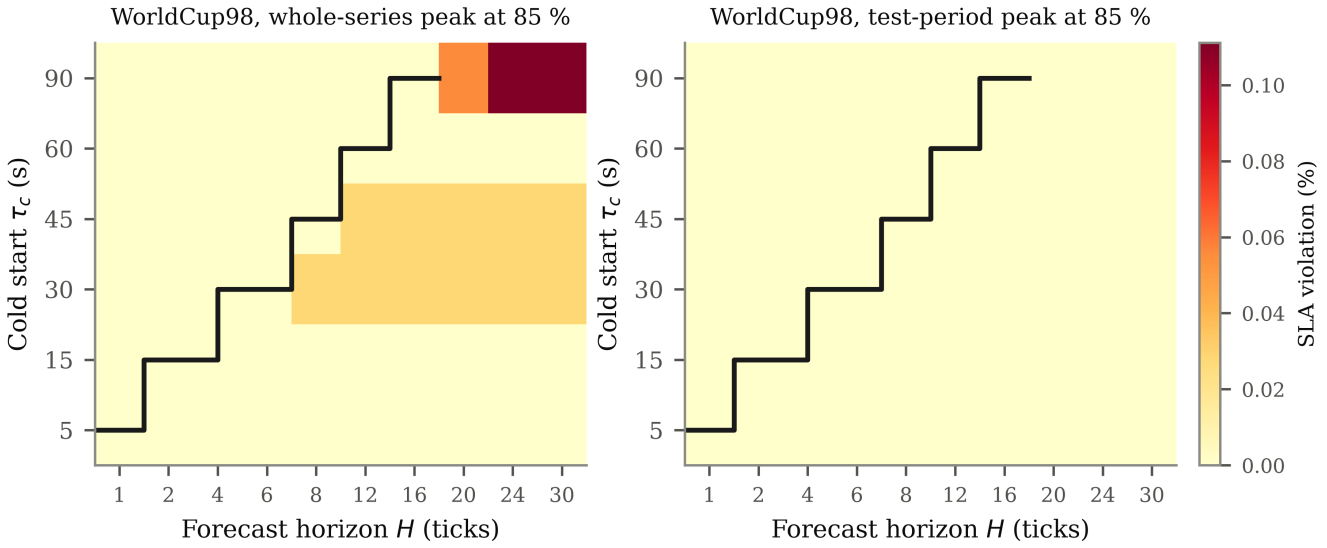


Fig. 21. Violation rate over the cold-start × horizon grid on recorded traffic, one panel per scaling. The line is HΔt = τc. Oracle predictor, 9 test windows per cell.

The integrality comparison of Section V-B was run on the recorded families alongside the synthetic ones. On WC98/trace the exact and rounded controllers disagree on 23.4 % of ticks and the rounded controller issues 30.5 actions per run against 52.4, with a violation difference of 1.98 points. On WC98/test the exact and rounded controllers disagree on 40.8 % of ticks and the rounded controller issues 114.1 actions per run against

44.0, with a violation difference of 1.38 points. On the synthetic families the corresponding figures are 38.4 %, 79.8 and 47.3.

The adaptive margin and the safety ceiling were examined in every recorded run. On WC98/trace, under the trained forecaster the margin moved in 63 of 86 runs (max 0.22) and under persistence it moved in 74 of 86 runs (max 0.18); the ceiling issued none in 86 runs under the trained forecaster and none in 86 runs under the oracle. On WC98/test, under the trained forecaster the margin moved in 86 of 86 runs (max 0.50) and under persistence it moved in 86 of 86 runs (max 0.50); the ceiling issued none in 86 runs under the trained forecaster and none in 86 runs under the oracle. In the forecast-corruption sweep of Section V-F repeated on recorded traffic, the ceiling issued 0 overrides in total on WC98/trace, and the curves with and without it coincide, and 0 overrides in total on WC98/test, and the curves with and without it coincide.

Finding	Synthetic	WC98/trace	WC98/test
Violation falls across $H\Delta t = \tau c$ (E5, E18)	holds	not testable (reactive never violates; ≤ 4 ticks per cell)	not observable
Short horizon worse than reactive (E5, E18)	holds (5 of 5)	not testable	not observable
Exact and rounded decisions differ (E3)	holds	holds	holds
Rounding raises churn (E3)	holds	fails	holds
Margin moves, oracle (E4 full)	never moved (50 runs)	never moved (10 runs)	moved in 3 of 10 runs (max 0.50)
Margin moves, persistence (E1)	moved in 50 of 50 runs (max 0.50)	moved in 8 of 10 runs (max 0.12)	moved in 10 of 10 runs (max 0.22)
Safety ceiling activates (E4 full)	160 overrides in 19 of 50 runs	none in 10 runs	none in 10 runs

Table XXI. Findings re-tested on recorded traffic. A transition "holds" when violation is lower above the boundary than below it at every cold-start setting sampled on both sides; "short horizon worse" holds when the predictive controller is worse than reactive below the boundary at one or more of those settings, with the count given; "not observable" means no cell produced a violation. The margin and ceiling rows count runs, not verdicts, because the earlier claim that both were inert is itself corrected in Section V-E.

What moves the trained arm

The TCN over-forecasts on average (Table XIX), so one candidate explanation for the trained arm's position is that the bias alone acts as a capacity margin. This was tested directly rather than by re-tuning the model: the oracle was shifted upward by the TCN's mean bias over all horizons, 2.15 req/s on WC98/trace and 8.65 req/s on WC98/test after the same rescaling, and run on the same windows. Table XXII compares it with the unshifted oracle and the trained arm. For violation, over-provisioning, cost and tail latency the shifted oracle moves in the same direction as the trained arm in 14 of 14 comparisons, and its mean lies closer to the trained arm's than the unshifted oracle's does in 14 of them: the bias accounts for the capacity-side behaviour. For scaling actions and oscillations it agrees in direction in only 4 of 8, and it does not reproduce the largest effect in Table XX, the rounded controller's churn: on WC98/trace the shifted oracle gives 45.3 actions per run against 42.7 unshifted and 22.2 trained, and on WC98/test the shifted oracle gives 148.3 actions per run against 151.6 unshifted and 72.5 trained. Some other property of the trained forecast damps the rounded controller. The smoothing effect measured in Section V-M is a candidate, but it was not tested here, and the paper does not claim it.

Family	Controller	Metric	Oracle	Oracle + TCN bias	Trained
WC98/trace	Exact	Violation (%)	0.02	0.00	0.00
WC98/trace	Exact	Over-prov. (%)	32.62	36.88	37.35
WC98/trace	Exact	Actions	50.95	51.64	50.43
WC98/trace	Rounded	Violation (%)	0.20	0.10	0.05
WC98/trace	Rounded	Over-prov. (%)	2.34	4.70	3.80
WC98/trace	Rounded	Actions	42.74	45.28	22.15
WC98/test	Exact	Violation (%)	0.00	0.00	0.00
WC98/test	Exact	Over-prov. (%)	11.23	14.33	13.30
WC98/test	Exact	Actions	48.78	47.86	45.58
WC98/test	Rounded	Violation (%)	0.03	0.03	0.03
WC98/test	Rounded	Over-prov. (%)	2.32	4.61	3.80
WC98/test	Rounded	Actions	151.63	148.30	72.50

Table XXII. The oracle shifted by the TCN's mean forecast bias, on the same 86 test windows per family as Table XX. Three of the six metrics are shown; results/e21_bias_probe.csv has all six.

Section V-E ablated the margin and the ceiling only under perfect foresight, where the margin barely moves. The exact controller was therefore re-run on every recorded test window with each mechanism disabled in turn, under the persistence and the trained predictors, with paired Wilcoxon tests Holm-corrected across all 24 comparisons (Table XXIII). The full-controller runs reproduce the corresponding means in Table XX. On WC98/trace under the persistence predictor the margin moved in 74 of 86 runs and the ceiling issued 0 overrides; disabling the margin changed the outcome in 47 windows and disabling the ceiling in 0. On WC98/trace under the trained TCN the margin moved in 63 of 86 runs and the ceiling issued 0 overrides; disabling the margin changed the outcome in 43 windows and disabling the ceiling in 0. On WC98/test under the persistence predictor the margin moved in 86 of 86 runs and the ceiling issued 0 overrides; disabling the margin changed the outcome in 86 windows and disabling the ceiling in 0. On WC98/test under the trained TCN the margin moved in 86 of 86 runs and the ceiling issued 0 overrides; disabling the margin changed the outcome in 79 windows and disabling the ceiling in 0. Of these differences, 4 are significant after correction: margin off, WC98/trace, persistence predictor, over-prov. -1.29; margin off, WC98/trace, persistence predictor, actions +0.70; margin off, WC98/test, persistence predictor, over-prov. -2.43; margin

off, WC98/test, trained TCN, over-prov. -1.15. The full controller violates on at most 0.006 % of ticks here, so these tests have almost no room to show a service-level benefit; what they do show is the capacity each mechanism holds.

Family	Predictor	Disabled	Δ violation (pp)	Δ over-prov. (pp)	Δ actions
WC98/trace	persistence predictor	margin	-0.003 (1.000)	-1.29 (0.021)	+0.7 (0.003)
WC98/trace	persistence predictor	ceiling	0 (identical)	0 (identical)	0 (identical)
WC98/trace	trained TCN	margin	-0.003 (1.000)	-0.66 (1.000)	+0.1 (0.438)
WC98/trace	trained TCN	ceiling	0 (identical)	0 (identical)	0 (identical)
WC98/test	persistence predictor	margin	0 (identical)	-2.43 (< 0.001)	+2.1 (0.064)
WC98/test	persistence predictor	ceiling	0 (identical)	0 (identical)	0 (identical)
WC98/test	trained TCN	margin	0 (identical)	-1.15 (< 0.001)	-1.9 (1.000)
WC98/test	trained TCN	ceiling	0 (identical)	0 (identical)	0 (identical)

Table XXIII. Exact MPC with the adaptive margin or the safety ceiling disabled, against the full controller, under the two imperfect predictors on recorded traffic, 86 test windows per family. Each cell is the mean paired difference with its Holm-corrected p-value; "identical" means no window changed.

VI. DISCUSSION

A. What the Measurements Support

Three claims survive the full set of experiments, and it is worth separating them from the claims the design anticipated but the data did not sustain.

The horizon condition is the strongest result and the one with the clearest practical reading. It holds at 5 of the 5 cold-start settings that straddle the boundary, the transition lands where (6) predicts rather than at a fixed horizon, and the sign of the predictive controller's advantage reverses across it. It also requires no tuning to exploit: the operator measures one latency and computes one inequality. Section V-J adds that once the horizon is adequate, the predictive controller is nearly insensitive to the delay itself over a factor of twelve, although at the longer delays of that sweep it ran on the greedy completion rather than the exact solve. It is, however, a result about the synthetic families and the simulator: on the recorded traffic at the scalings used, the reactive baseline never violated, so the transition could not be tested there (Section V-Q).

The integrality result is narrower. Exact and rounded controllers differ on a large and growing fraction of decisions, from 18.8 % to 63.6 % across the pool bounds of Table V. At the larger pools the difference materialises as churn, at aggressive operating points as a service collapse (Table VI), and at the default operating point the violation difference is not significant after correction ($p = 0.103$) while the churn difference is unambiguous (Cliff's $\delta = -0.88$, $p < 0.001$). The churn advantage is not universal: it reverses at the smallest pools, and on recorded traffic it holds on WC98/test only.

The third claim is about stability mechanisms, and it inverts part of the design's assumption. The cooldown, which was included as a conventional guard rail, turned out to dominate: removing it raises churn from 44.5 to 272.1 actions per run. The churn penalty in the objective matters too, and raising it from 0 to 1 improves service level and cost together rather than trading between them. The deadband is inert at its default; the cost asymmetry is not, and raising it from 1 to 20 changes violations from 0.85 % to 0.39 %.

B. What the Measurements Do Not Support

Several anticipated effects were absent or smaller than designed, and reporting them is as informative as reporting the successes because each identifies either a redundant mechanism or an untested one.

The explicit dead-time term in the plant model bought no detectable service level when the horizon was already adequate, and it cost capacity: disabling it lowered over-provisioning by 16.9 points and churn by 19.9 actions (Table IX). Dead-time tolerance is therefore attributed to the predictive formulation as a whole, not to the delay term.

The adaptive margin and the safety ceiling are active but of unestablished value. The margin moves under every imperfect predictor tested and barely at all under perfect foresight; the ceiling fires in the main comparison, the ablation and the corruption sweep. Disabling either one, under perfect foresight (Table IX) or under the persistence and trained predictors (Table XXIII), produced no significant change in the violation rate. An earlier version of this paper called both inert. The result files of that version already showed the ceiling firing; they did not record the margin at all, so its claim had no measurement behind it. Both claims are withdrawn.

The trained forecaster did not land between the brackets as the protocol assumes: it was inside a non-degenerate bracket on 6 of 23 comparisons (Table XX). Its upward bias explains the capacity-side departures (14 of 14 in Table XXII) but not the rounded controller's reduced churn, so the bracketing protocol should be read as a guide to the predictor's effect, not a bound on it.

Finally, the proposed controller did not dominate. It carries more capacity than the rounded controller at matched settings (23.6 % against 3.2 %) and issues more scaling actions than the reactive baseline (44.4 against 12.1). An operator whose workload resembles the slow families in Table XIV, and who can tolerate the rounded controller's violation rate, has no reason to prefer exact solution on these results.

C. When Each Mechanism Is Worth Its Cost

Synthesising across the experiments gives a decision rule that is narrower than a general recommendation but better supported than one. Exact integer solution is worth adopting when the pool is small relative to demand swings, when the target utilisation is aggressive, or when the workload has fast transitions — the three conditions under which Table V, Table VI and Table XIV respectively show the largest separation. It is not worth adopting on slowly varying traffic at conservative targets, where the measurements show no advantage and a capacity penalty.

Predictive scaling in general is worth adopting when the actuation delay is long relative to the control interval, which Section V-J quantifies, and only when the horizon can be

configured to cover it. Where the delay is short — at τc = 5 s the reactive baseline violates on 0.17 % of ticks against 0.05 % predictive — the additional machinery buys little.

VII. LIMITATIONS AND THREATS TO VALIDITY

A. Construct Validity

Construct validity asks whether the quantities measured correspond to the properties claimed. Two constructs in this paper are imperfect and the imperfection bounds the claims that rest on them.

The service-level violation rate counts ticks whose modelled tail latency exceeds the objective. Real service-level objectives are evaluated over windows and against measured request latencies, not over control ticks against a queueing approximation. A controller that performs well on this construct would perform well on the real one only to the extent the approximation holds, which Section IV-B argues it does in the moderate-utilisation regime and does not near saturation, where the measure is censored.

The ground-truth replica requirement, used by both the decision classification of Section V-L and the excess column of Table XVII, is computed from the instantaneous arrival rate. It therefore treats noise as demand and changes on nearly every tick. It is an adequate construct for measuring systematic bias, which is how this paper uses it, and an inadequate one for ranking controllers by decision quality, which is why the classification results are reported but not used to support any claim.

B. Internal Validity

Internal validity asks whether the measured differences are caused by the manipulated variable. The experimental design supports this reasonably well. Every method sees the identical trace for a given family and seed, so comparisons are paired and the Wilcoxon test is appropriate. Each ablation and sweep changes exactly one component or parameter with the others held fixed. The plant is shared across methods within a run, so differences cannot arise from plant variation.

One threat remains. The methods operate at different default utilisations, so the aggregate comparison of Table VII compares points rather than frontiers. This is the reason the operating-point sweep of Section V-C exists, and conclusions about service level are drawn from that sweep rather than from the aggregate table.

C. External Validity

External validity asks how far the results transfer, and it is where this evaluation is weakest. The plant is simulated; five of the seven workload families are synthetic and the other two are rescaled copies of one recorded trace; one forecaster was trained, once; and a single replica pool was controlled. Each is enumerated in the following subsection with the specific claim it bounds.

One form of transfer is better supported than the others. The horizon condition of (6) is a statement about the relationship between two latencies, and the experiment confirms the transition occurs where that relationship predicts rather than at a fixed horizon. A result that tracks a parameter in this way is more likely to transfer than one that holds only at the values tested, because the mechanism rather than the magnitude is what has been identified. The integrality result is in the opposite position: the magnitude of the rounding gap depends on pool size, demand volatility and the operating point, and Table V shows the share of ticks on which the two controllers disagree varying by a factor of 3.4 across pool sizes alone.

D. Enumerated Limitations

This evaluation has substantial limitations. They are listed here rather than distributed through the text because several of them bound the claims directly.

Simulated plant only. No result in this paper was obtained on real containers. The plant is the analytical queueing simulator described in Sections IV-A and IV-B, and the cold-start interval is a parameter of that simulator rather than a measurement. Image pull variance, scheduler interference, connection draining and load-balancer registration delay are all absent. The horizon–dead-time law of Section V-A is a property of the model as much as of the controller, and confirming it on a real Docker deployment is the single most important piece of outstanding work.

One recorded trace, rescaled. Five of the seven workload families are generated rather than recorded. The two recorded families are one trace, days 41–50 of the WorldCup98 log, at two scalings, and a scaling preserves the shape of the traffic but not its absolute level: real request rates were divided by a constant to fit a sixteen-replica pool, so the per-replica variability is not that of the original servers. The second scaling was chosen from the peak of the test split, which uses one statistic of the test data; the forecaster's inputs are standardised with training statistics, under which a constant factor cancels, but the choice of operating point is not blind. The test split is a single weekend, the trace contains a short dip at each local midnight that was left unrepaired, and a second recorded source (Bitbrains GWA-T-12) was located but not used because it records per-machine utilisation at five-minute intervals rather than request arrivals.

The start state was changed after a diagnosis. Runs originally opened with four replicas. On recorded windows that placed every violation in the start-up transient, so the rule was changed to the larger of four and the first tick's requirement and every experiment was re-run (Section V-Q). The change was made before any trained-arm result existed and applies to every workload, but it is a change of protocol made after looking at data, and it is reported as one.

One forecaster, trained once. The trained forecaster is a single TCN checkpoint from one seed, and each baseline was also trained once, so the ranking in Table XIX carries no estimate of training variance. The TCN ranked 6 of 7 on accuracy at the full horizon, behind ARIMA; a better-tuned model might change where the trained arm falls relative to the brackets. It was trained on the MPS device of one workstation and evaluated on its CPU. The trained arm appears only in the comparison of Table XX: the horizon sweeps and the corruption sweep use the oracle, because the checkpoint has a fixed horizon and because those experiments isolate the controller by design.

Two mechanisms are only partly tested. An earlier version of this paper stated that the adaptive margin never moved and the safety ceiling never activated. Both statements were wrong. The margin moves under every imperfect predictor tested (Sections V-E and V-Q), and the ceiling fired in the main comparison, the ablation and the corruption sweep. What remains untested is their value: no experiment here shows either one improving the service level significantly, including the ablation under imperfect predictors in Table XXIII, and the corruption model is symmetric, so it never produces the sustained under-prediction the ceiling exists to catch.

Censored latency. The heavy-traffic approximation diverges when offered load exceeds pool capacity. Such ticks are censored at a finite ceiling, which compresses the upper tail of the latency distribution and makes the reported mean latency a lower bound in saturated conditions.

Single service. Only one replica pool is controlled. Real deployments comprise interdependent services sharing hosts, and the coupling between their scaling decisions is not modelled.

Baseline fidelity. The relaxed-and-rounded controller reproduces the integrality choice made by published optimisation-based auto-scalers on an otherwise identical objective; it is not a reimplementation of any specific published system, and it omits the chance constraints that some of them use. It should be read as an ablation of the integrality constraint rather than as a competing system.

VIII. CONCLUSION

Three properties of predictive container auto-scaling were measured on a simulated pool, with the predictor held at two extremes so that the control law was the only variable. They differ in how firmly they are established, and the distinction is worth keeping.

The forecast horizon must cover the actuation dead time. Across the 5 cold-start settings sampling both sides of the boundary, the violation rate fell from 2.92 % to 0.27 % as the horizon crossed $H\Delta t = \tau c$, and below that boundary the predictive controller performed worse than a reactive baseline by up to 3.69 percentage points. Horizon selection is therefore not a tuning preference but a feasibility condition, and it can be checked before deployment from a single measurement of cold-start latency. This is the firmest of the three results on the synthetic families: the transition appeared at every cold-start setting tested and landed where the inequality predicts rather than at a fixed horizon. On the recorded traffic at the scalings used, the reactive baseline never violated, so the transition could not be tested there (Section V-Q).

The integer problem does not have to be approximated. Dead time confines the tracking-relevant decisions to the first $H - \tau + 1$ steps of the horizon and leaves the rest separable, which reduced the search to 4 nested integer choices at the configuration used here. The solver built on that reduction reproduced the brute-force optimum in all 400 states small enough to enumerate exhaustively, which is a verification rather than a proof but a stronger warrant than the argument it replaces. Completing the horizon greedily instead, a device sometimes called exact on the grounds that only the first action is applied, returned an optimal first action in only 354 of those 400 states, and at the paper configuration differed from the optimum on 10.83 % of the plant states drawn, always in the direction of higher cost.

Relaxing the integrality of the scaling decision changes behaviour measurably. The relaxed controller disagreed with the exact one on 18.8 to 63.6 % of control ticks, issued up to 3.8 times as many scaling actions, and degraded from 0.32 % to 16.18 % violations as the utilisation target was raised, against 0.00 % to 2.86 % for the exact controller. Exact solution cost at most 17.54 ms per tick, 0.35 % of the control interval, so relaxation buys compute that is not scarce at these problem sizes. That is a claim about this configuration rather than a general one: the exact solve is cheap here because the dead time is long relative to the horizon, and a pool with near-instant start-up and a long horizon would not enjoy the same reduction.

The results also qualify the approach. At the default operating point the exact controller did not significantly reduce violations relative to either the rounded or the reactive baseline, and it carried more capacity than the rounded controller to achieve the service level it did. Its measured advantages are in actuation stability at the larger pool bounds and in graceful degradation at aggressive operating points. Its adaptive margin and safety ceiling are active, contrary to an earlier version of this paper, but no experiment here establishes that either improves the service level, and its dead-time term costs capacity without a measured service-level return.

The immediate priorities for further work follow from Section VII: reproducing the horizon–dead-time transition on a real Docker deployment with a measured cold-start interval; repeating the trained-forecaster comparison with several training seeds, several model families and more than one recorded trace; identifying what in the trained forecast damps the rounded controller; and designing a corruption model with sustained bias that would exercise the margin adaptation and the safety ceiling.

REFERENCES

[1] T. Van Do et al., "Properties of horizontal pod autoscaling algorithms and application for scaling cloud-native network functions," preprint, 2025.

[2] B. Jeong and Y.-S. Jeong, "ARAScaler: Adaptive resource autoscaling scheme using ETimeMixer for efficient cloud-native computing," IEEE Trans. Services Computing, vol. 18, no. 1, pp. 72–84, 2025, doi: 10.1109/TSC.2024.3522815.

[3] J. W. Fog, J. J. T. Møller, T. M. Jensen, D. Taibi, and M. Albano, "Comparing neural and statistical time-series models for proactive auto-scaling in Kubernetes," in Proc. IEEE Int. Conf. Service-Oriented System Engineering (SOSE), 2025, pp. 151–161, doi: 10.1109/SOSE67019.2025.00022.

[4] E. Incerto, M. Tribastone, and C. Trubiani, "Combined vertical and horizontal autoscaling through model predictive control," in Euro-Par 2018: Parallel Processing, Lecture Notes in Computer Science, vol. 11014, Springer, 2018, pp. 147–159, doi: 10.1007/978-3-319-96983-1_11.

[5] G. Quattrocchi, E. Incerto, R. Pinciroli, C. Trubiani, and L. Baresi, "Autoscaling solutions for cloud applications under dynamic workloads," IEEE Trans. Services Computing, vol. 17, no. 3, pp. 804–820, 2024, doi: 10.1109/TSC.2024.3354062.

[6] D. Zou, W. Lu, Z. Zhu, X. Lu, J. Zhou, X. Wang, K. Liu, K. Wang, R. Sun, and H. Wang, "OptScaler: A collaborative framework for robust autoscaling in the cloud," Proc. VLDB Endowment, vol. 17, no. 12, pp. 4090–4103, 2024, doi: 10.14778/3685800.3685829.

[7] Z. Cai et al., "Deep learning and feedback control based container auto-scaling for cloud native microservices," IEEE Trans. Services Computing, 2025.

[8] F. Meng et al., "CATScaler: A convolution-augmented transformer scaling framework for cloud-native applications," IEEE Trans. Services Computing, 2025.

[9] Z. Fang, H. Ma, G. Chen, and R. Buyya, "HGraphScale: Hierarchical graph learning for autoscaling microservice applications in container-based cloud computing," IEEE Trans. Services Computing, 2025.

[10] B. Kumar, A. Verma, and R. Buyya, "Predictive autoscaling in cloud-native and federated cloud-edge computing environments: A taxonomy and future directions," arXiv:2606.07046, 2026.

[11] M. Arlitt and T. Jin, "1998 World Cup Web Site Access Logs", August 1998. Available at http://www.acm.org/sigcomm/ITA/.